

学 号： 2022212357

密 级： 公开

合肥工业大学

Hefei University of Technology

本科毕业设计（论文）

UNDERGRADUATE THESIS



类 型：	设计
题 目：	可证安全图像隐写系统设计与实现
专业名称：	信息安全
入校年份：	2022 级
学生姓名：	支雅安
指导教师：	王垚飞 副教授
学院名称：	计算机与信息学院
完成时间：	2026 年 5 月

合 肥 工 业 大 学

本科毕业设计（论文）

可证安全图像隐写系统设计与实现

学生姓名： 支雅安

学生学号： 2022212357

指导教师： 王垚飞 副教授

专业名称： 信息安全

学院名称： 计算机与信息学院

2026 年 5 月

A Dissertation Submitted for the Degree of Bachelor

**Design and Implementation of a Provably Secure
Image Steganography System**

By

ZHI Ya'an

Hefei University of Technology

Hefei, Anhui, P.R.China

May, 2026

毕业设计（论文）独创性声明

本人郑重声明：所提交的毕业设计（论文）是本人在指导教师指导下进行独立研究工作所取得的成果。据我所知，除了文中特别加以标注和致谢的内容外，设计（论文）中不包含其他人已经发表或撰写过的研究成果，也不包含为获得 合肥工业大学 或其他教育机构的学位或证书而使用过的材料。对本文成果做出贡献的个人和集体，本人已在设计（论文）中作了明确的说明，并表示谢意。

毕业设计（论文）中表达的观点纯属作者本人观点，与合肥工业大学无关。

毕业设计（论文）作者签名： 学生签 签名日期：2026 年 5 月 29 日

毕业设计（论文）版权使用授权书

本学位论文作者完全了解 合肥工业大学 有关保留、使用毕业设计（论文）的规定，即：除保密期内的涉密设计（论文）外，学校有权保存并向国家有关部门或机构送交设计（论文）的复印件和电子光盘，允许设计（论文）被查阅或借阅。本人授权 合肥工业大学 可以将本毕业设计（论文）的全部或部分内容编入有关数据库，允许采用影印、缩印或扫描等复制手段保存、汇编毕业设计（论文）。

（保密的毕业设计（论文）在解密后适用本授权书）

学位论文作者签名： 学生签

签名日期：2026 年 5 月 29 日

指导教师签名： 导师签

签名日期：2026 年 5 月 29 日

摘 要

随着网络通信技术的快速发展，信息安全与隐私保护日益受到重视。图像隐写术作为信息隐藏技术的重要分支，通过将秘密信息嵌入载体图像中实现隐蔽通信，在国家安全、版权保护和隐私通信等领域具有重要应用价值。传统隐写方法在安全性方面存在不足，容易被基于深度学习的隐写分析工具检测。近年来，基于生成模型的可证安全隐写方案为解决这一问题提供了理论保障。

本文设计并实现了一个基于扩散模型的可证安全图像隐写通信系统。系统集成了两种可证安全隐写算法：**Pulsar** 算法利用 **DDPM** 的逆向采样过程，结合纠错编码与伪随机数生成器，将秘密消息编码为采样路径；**SparSamp** 算法基于稀疏采样思想，通过消息驱动采样将消息段与伪随机数结合生成消息导出随机数（MRN），在 **IDDPM** 的 **P2-weighting** 框架下逐像素采样嵌入消息，并通过稀疏采样机制消除解码歧义。两种算法均保证含密图像与正常生成图像在统计分布上不可区分。

在通信安全方面，系统实现了基于对称助记词的端到端加密机制。通信双方各自持有一个助记词短语，通过 **PBKDF2-SHA256** 派生用户密钥；双方的用户密钥经 **HKDF-SHA256** 生成对称消息密钥，使用 **AES-256-GCM** 对消息内容进行加密封装。隐写聊天模式下，加密后的密文作为隐写嵌入的载荷，实现了“先加密、再隐写”的双重保护。密钥协商采用基于邀请码的 **XOR** 方案，通信双方无需预先交换密钥即可独立计算出相同的隐写种子。

系统采用客户端-服务端架构，服务端基于 **Python FastAPI** 框架构建，集成双算法隐写引擎和 **WebSocket** 实时通信服务；客户端包括基于 **Kotlin Jetpack Compose** 的 **Android** 应用和基于 **Vue.js 3** 的 **Web** 应用，均实现了用户注册登录、好友管理、实时聊天（支持消息送达/已读回执）、隐写消息收发和端到端加密配置等完整功能。**Web** 端采用 **Aegis Slate** 暗色 **Material 3** 设计系统，**Android** 端采用 **Material 3 Expressive** 薄荷色主题。

测试结果表明，系统功能完整、运行稳定，能够正确实现消息的端到端加密与隐写嵌入提取，双算法切换正常，跨平台通信兼容性良好。本系统将可证安全隐写理论与端到端加密相结合，为安全隐蔽通信提供了一种可行的工程实现方案。

关键词：图像隐写；可证安全；扩散模型；端到端加密；跨平台通信；隐写系统

ABSTRACT

With the rapid development of network communication technologies, information security and privacy protection have attracted increasing attention. Image steganography, as an important branch of information hiding, achieves covert communication by embedding secret messages into cover images and has significant applications in national security, copyright protection, and private communication. Traditional steganographic methods suffer from security deficiencies and are susceptible to detection by deep learning-based steganalysis tools. In recent years, provably secure steganographic schemes based on generative models have provided theoretical guarantees for addressing this problem.

This thesis designs and implements a provably secure image steganography communication system based on diffusion models. The system integrates two provably secure steganographic algorithms: the Pulsar algorithm, which leverages the reverse sampling process of DDPM combined with error-correcting codes and pseudo-random number generators to encode secret messages as sampling paths; and the SparSamp algorithm, based on the concept of sparse sampling, which combines message segments with pseudo-random numbers via modular addition to generate Message-derived Random Numbers (MRNs) for pixel-wise sampling under the IDDPM P2-weighting framework, and resolves decoding ambiguity through iterative candidate set narrowing. Both algorithms guarantee that stego images are statistically indistinguishable from normally generated images.

For communication security, the system implements a symmetric-mnemonic-based end-to-end encryption mechanism. Each party holds a mnemonic phrase from which a user key is derived via PBKDF2-SHA256; the two user keys are then combined through HKDF-SHA256 to produce a symmetric message key, which is used with AES-256-GCM to encrypt and seal message content. In steganographic chat mode, the encrypted ciphertext serves as the payload for steganographic embedding, achieving a dual protection of “encrypt first, then hide.” The key negotiation uses an XOR scheme based on invite codes, allowing both parties to independently compute the same steganographic seed without prior key exchange.

The system adopts a client-server architecture. The server is built on the Python FastAPI

framework, integrating a dual-algorithm steganographic engine and WebSocket real-time communication services. The client side includes an Android application based on Kotlin Jetpack Compose and a Web application based on Vue.js 3, both implementing complete functionalities including user registration and login, friend management, real-time chat with message delivery and read receipts, steganographic message transmission, and end-to-end encryption configuration. The Web client adopts the Aegis Slate dark Material 3 design system, while the Android client uses a Material 3 Expressive mint-themed design.

Testing results demonstrate that the system is functionally complete and operates stably, correctly performing end-to-end encryption and steganographic embedding/extraction with seamless dual-algorithm switching and good cross-platform communication compatibility. This system combines provably secure steganography theory with end-to-end encryption, providing a feasible engineering solution for secure covert communication.

KEYWORDS: Image Steganography; Provable Security; Diffusion Model; End-to-End Encryption; Cross-platform Communication; Steganography System

目 录

1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 传统图像隐写方法	2
1.2.2 基于深度学习的隐写分析	2
1.2.3 可证安全隐写	3
1.2.4 端到端加密技术	3
1.3 主要研究内容	4
1.4 论文组织结构	5
2 相关技术与理论基础	6
2.1 图像隐写术概述	6
2.1.1 空间域隐写	6
2.1.2 变换域隐写	6
2.1.3 基于生成模型的隐写	6
2.2 可证安全隐写理论	7
2.2.1 信息论安全模型	7
2.2.2 Cachin 安全准则	7
2.3 Pulsar 算法原理	7
2.3.1 扩散模型基础	8
2.3.2 Pulsar 隐写编码原理	8
2.3.3 Pulsar 消息提取	9
2.3.4 纠错编码机制	10
2.4 SparSamp 算法原理	10
2.4.1 P2-weighting 框架	11

2.4.2 SparSamp 嵌入流程	11
2.4.3 SparSamp 提取流程	12
2.4.4 状态缓存机制	12
2.5 端到端加密机制	12
2.5.1 用户密钥派生	12
2.5.2 消息密钥协商	13
2.5.3 消息加密封装	14
2.5.4 隐写密钥协商	14
2.5.5 隐写聊天的双重保护	14
2.6 关键技术栈	16
2.6.1 FastAPI 框架	16
2.6.2 WebSocket 协议	16
2.6.3 Jetpack Compose	16
2.6.4 Vue.js 3	16
2.6.5 本地存储技术	17
2.6.6 密码学库	17
2.7 本章小结	17
3 系统设计	18
3.1 需求分析	18
3.1.1 功能需求	18
3.1.2 非功能需求	18
3.2 系统总体架构	19
3.3 服务端设计	19
3.3.1 API 接口设计	19
3.3.2 数据库设计	21
3.3.3 WebSocket 通信协议设计	21
3.3.4 消息生命周期设计	23

3.3.5	离线消息队列设计	23
3.3.6	双算法隐写服务设计	23
3.4	客户端设计	24
3.4.1	Android 客户端架构	24
3.4.2	Web 客户端架构	24
3.4.3	本地存储设计	25
3.5	安全设计	25
3.5.1	威胁模型	25
3.5.2	JWT 认证机制	26
3.5.3	密码安全	26
3.5.4	端到端加密设计	26
3.5.5	隐写密钥派生	27
3.5.6	单端登录策略	27
3.6	本章小结	27
4	系统实现	31
4.1	服务端实现	31
4.1.1	FastAPI 应用结构	31
4.1.2	双算法隐写服务集成	31
4.1.3	WebSocket 连接管理	32
4.1.4	离线消息队列	33
4.2	Android 客户端实现	33
4.2.1	界面实现	33
4.2.2	Room 数据库实现	34
4.2.3	端到端加密实现	34
4.2.4	WebSocket 客户端实现	35
4.2.5	隐写功能实现	35
4.3	Web 客户端实现	36

4.3.1	Vue 组件结构	36
4.3.2	Pinia 状态管理	36
4.3.3	IndexedDB 本地存储	37
4.3.4	端到端加密实现	37
4.3.5	WebSocket 与隐写功能	37
4.4	关键功能实现细节	38
4.4.1	单端登录机制	38
4.4.2	好友请求流程	38
4.4.3	隐写消息收发流程	38
4.5	本章小结	40
5	系统测试与分析	43
5.1	测试环境	43
5.2	功能测试	43
5.2.1	用户注册与登录测试	43
5.2.2	好友管理测试	43
5.2.3	消息收发与生命周期测试	46
5.2.4	端到端加密测试	46
5.2.5	隐写嵌入与提取测试	48
5.3	隐写性能分析	49
5.3.1	算法性能对比	49
5.3.2	安全性分析	51
5.3.3	图像质量	51
5.4	跨平台兼容性测试	53
5.5	本章小结	53
6	总结与展望	55
6.1	工作总结	55
6.2	不足与展望	56

参考文献	58
致谢	60
附录	61

插图清单

图 2.1 扩散模型前向加噪与逆向去噪过程	8
图 2.2 Pulsar 隐写嵌入流程	9
图 2.3 Pulsar 消息提取流程	10
图 2.4 SparSamp 嵌入流程	12
图 2.5 E2EE 密钥派生层次	13
图 2.6 加密-隐写双重保护流程	15
图 3.1 系统总体架构	20
图 3.2 WebSocket 通信时序	28
图 3.3 消息状态机	29
图 3.4 Android 客户端 MVVM 架构	30
图 3.5 Web 客户端架构	30
图 4.1 单端登录时序	39
图 4.2 好友请求流程	41
图 4.3 隐写消息收发时序	42
图 5.1 用户注册与登录界面	45
图 5.2 好友管理界面	46
图 5.3 消息收发界面	47
图 5.4 端到端加密功能	48
图 5.5 隐写嵌入与提取功能	48
图 5.6 Pulsar 与 SparSamp 性能对比	50
图 5.7 隐写样本对比	52
图 5.8 隐写模式聊天效果	52
图 5.9 单端登录效果	54

表格清单

表 3.1	服务端 API 接口设计	21
表 3.2	服务端数据库表结构.....	22
表 3.3	WebSocket 消息类型	22
表 4.1	Web 客户端页面组件	36
表 5.1	测试环境配置.....	43
表 5.2	用户注册与登录测试用例.....	44
表 5.3	好友管理测试用例	44
表 5.4	消息收发与生命周期测试用例	46
表 5.5	端到端加密测试用例.....	47
表 5.6	隐写嵌入与提取测试用例.....	49
表 5.7	Pulsar 与 SparSamp 算法性能对比.....	49
表 5.8	含密图像与原始生成图像的质量对比.....	51
表 5.9	跨平台兼容性测试	53

1 绪论

1.1 研究背景与意义

随着互联网技术的飞速发展和数字化进程的深入推进，网络已经成为人们日常生活中不可或缺的信息交流渠道。然而，网络通信的开放性和共享性也带来了严峻的信息安全挑战。政府、企业和个人的敏感信息在传输过程中面临着被窃取、篡改和监控的风险^[1]。传统的加密技术虽然能够保护信息内容的机密性，但加密通信行为本身容易引起监控者的注意，从而招致进一步的审查和干预。

信息隐藏技术（Information Hiding）为解决上述问题提供了一种独特的思路。与加密技术不同，信息隐藏并不改变通信形式的外观，而是将秘密信息嵌入到看似正常的载体媒体（如图像、音频、视频等）中，使得通信双方的交互在外部观察者看来与正常通信无异^[2]。图像隐写术（Image Steganography）作为信息隐藏技术的重要分支，因其载体图像在网络中传输量大、格式多样、视觉冗余丰富等特点，成为研究最为活跃的方向之一。

早期的图像隐写方法主要基于空间域修改策略，如最低有效位（Least Significant Bit, LSB）替换^[3]。这类方法实现简单，但嵌入操作会不可避免地改变载体图像的统计特性，容易被统计隐写分析方法检测^[4]。随着深度学习技术的发展，基于卷积神经网络的隐写分析工具（如 SRNet^[5]）展现出了极强的检测能力，对传统隐写方法构成了严重威胁。

为了从根本上解决隐写安全性问题，研究者提出了可证安全隐写（Provably Secure Steganography）的理论框架^[6]。该理论指出，如果隐写系统生成的含密载体分布与正常载体分布在统计上不可区分（即 KL 散度为零），则任何检测器都无法以优于随机猜测的概率检测出隐写行为。近年来，基于深度生成模型（如变分自编码器 VAE、生成对抗网络 GAN、扩散模型 Diffusion Model）的隐写方案为实现可证安全提供了新的技术路径^[7]。

与此同时，端到端加密（End-to-End Encryption, E2EE）技术的发展为即时通信安全提供了另一层保障。端到端加密确保只有通信双方能够解密消息内容，即使服务端被攻破也无法获取明文信息。Signal 协议^[8]是目前最具代表性的端到端加密方案，被 WhatsApp、Signal 等主流通信应用广泛采用。将端到端加密与图像隐写相结合，可以实现“先加密、再隐写”的双重保护：即使隐写图像被提取出来，没有密钥

也无法解密得到原始消息内容。

本文的研究正是在上述背景下展开的。本文选择基于扩散模型的 **Pulsar** 算法^[9] 和 **SparSamp** 算法作为核心隐写方案，结合端到端加密机制，设计并实现一个完整的可证安全图像隐写通信系统。本文的工作目标包括以下两个方面：

1. **算法工程化验证**：将 **Pulsar** 和 **SparSamp** 两种可证安全隐写算法集成到统一的服务框架中，验证其在真实通信环境下的嵌入容量、提取正确率和计算效率，并通过端到端加密与隐写的组合实现双重安全保护。
2. **跨平台系统实现**：构建包含服务端 (**FastAPI**)、**Android** 客户端 (**Jetpack Compose**) 和 **Web** 客户端 (**Vue.js 3**) 的完整隐写通信系统，实现从用户注册、好友管理到隐写消息收发的全流程功能，为可证安全隐写从理论走向工程应用提供可参考的实现方案。

1.2 国内外研究现状

1.2.1 传统图像隐写方法

传统图像隐写方法可大致分为空间域方法和变换域方法两大类。

空间域隐写方法直接在图像像素值上进行修改。最经典的是 **LSB** 替换方法，通过将秘密信息的比特替换载体图像像素的最低有效位实现嵌入^[3]。为了减小嵌入对图像统计特性的影响，研究者提出了 **LSB 匹配**^[10]、**HUGO (Highly Undetectable Stego)**^[11]、**WOW (Wavelet Obtained Weights)**^[12] 和 **S-UNIWARD**^[13] 等自适应隐写方法，通过最小化嵌入失真函数来优化嵌入位置的选择。

变换域隐写方法在图像的频域或其他变换域中嵌入信息。典型的方法包括基于离散余弦变换 (**DCT**) 的 **JPEG** 域隐写^[14]，如 **F5** 算法、**nsF5** 算法^[15] 和 **J-UNIWARD**^[13] 等。

尽管上述方法在降低可检测性方面取得了显著进展，但由于嵌入操作本质上是对已有载体的修改，不可避免地会引入统计痕迹，无法在理论上保证绝对安全。

1.2.2 基于深度学习的隐写分析

隐写分析 (**Steganalysis**) 旨在检测载体中是否隐藏了秘密信息。传统隐写分析方法依赖手工设计的特征提取器，如 **SRM (Spatial Rich Model)**^[4]。随着深度学习的兴起，基于卷积神经网络的隐写分析方法展现了更优异的检测性能。代表性工作包

括 Qian-Net^[16]、Xu-Net^[17]、Ye-Net^[18] 和 SRNet^[5] 等。这些方法能够自动学习隐写特征，对多种隐写方案都具有较高的检测准确率，进一步凸显了传统隐写方法在安全性方面的不足。

1.2.3 可证安全隐写

可证安全隐写的理论基础由 Hopper 等人^[6] 奠定。其核心思想是：隐写系统的安全性可以归结为含密载体分布与正常载体分布之间的统计距离（如 KL 散度）。若两者分布完全一致，则隐写系统是完美安全的（Perfectly Secure）。Cachin^[19] 进一步从信息论角度形式化了这一安全准则。

实现可证安全的关键在于隐写编码过程不能对载体分布引入任何偏差。基于深度生成模型的隐写方案通过在模型的采样过程中嵌入信息，天然地满足了这一要求。代表性工作包括：

- 基于 GAN 的方案：利用 GAN 的随机输入向量（latent code）嵌入信息^[20]。
- 基于 VAE 的方案：在变分自编码器的隐变量采样过程中嵌入信息。
- 基于 Flow 的方案：利用可逆归一化流模型的采样过程嵌入信息^[7]。
- 基于扩散模型方案：利用扩散模型逆向去噪过程中方差噪声的随机性嵌入信息。Pulsar^[9] 是该方向的代表性工作，由 Jois 等人发表于 ACM CCS 2024，通过控制最后一步调度器的噪声来源实现逐比特编码，结合变码率纠错码在 256×256 分辨率下平均嵌入 320–613 字节。SparSamp^[21] 则基于稀疏采样思想，通过消息驱动采样将消息段与伪随机数结合生成 MRN，在 IDDPM 的 P2-weighting 框架下逐像素采样嵌入消息，并通过逐步缩小候选消息集合消除解码歧义，单图容量可达数十千字节。

与其他生成模型相比，扩散模型生成的图像质量更高、多样性更好，是当前最有前景的可证安全隐写载体。

1.2.4 端到端加密技术

端到端加密（E2EE）是保障即时通信安全的核心技术。其基本思想是：消息在发送方设备上加密，只有接收方设备能够解密，中间节点（包括服务端）只能转发密文而无法获取明文。

Signal 协议^[8] 是目前最成熟的端到端加密方案，采用双棘轮算法（Double Ratchet

Algorithm) 实现前向安全和未来安全性。然而, Signal 协议的完整实现复杂度较高, 需要维护大量的密钥状态和会话信息。

对于特定应用场景, 可以采用更轻量的对称加密方案。通过预共享的助记词短语派生用户密钥, 再基于通信双方的用户密钥协商出对称消息密钥, 使用 AES-GCM 认证加密算法保护消息内容。这种方案在安全性上虽不如 Signal 协议完备 (缺乏前向安全), 但实现简单、性能优异, 适合资源受限的移动端场景。

1.3 主要研究内容

本文围绕可证安全图像隐写系统的设计与实现, 开展了以下主要工作:

1. **双算法隐写引擎**: 集成 Pulsar 和 SparSamp 两种基于扩散模型的可证安全隐写算法, 设计了统一的算法注册与切换机制。Pulsar 基于 DDPM 框架, SparSamp 基于 IDDPM P2-weighting 框架, 两者共享相同的 API 接口但底层实现独立。
2. **端到端加密机制**: 实现了基于对称助记词的端到端加密方案。用户通过助记词短语经 PBKDF2-SHA256 派生 256 位用户密钥; 通信双方的用户密钥经 HKDF-SHA256 协商出对称消息密钥; 消息使用 AES-256-GCM 加密封装为 SealedMessage 格式。
3. **系统架构设计**: 设计了基于客户端-服务端的跨平台隐写通信系统架构, 服务端负责隐写算法计算和消息路由, 客户端 (Android 和 Web) 负责用户交互、本地数据管理和加解密操作。
4. **实时通信系统**: 基于 WebSocket 协议实现了服务端与客户端之间的实时双向通信, 支持消息即时推送、消息状态回执 (已发送/已送达/已读)、消息撤回、在线状态感知和离线消息缓存。
5. **用户体系与好友管理**: 实现了完整的用户注册登录系统 (JWT 认证)、基于邀请码的好友添加功能、好友请求的异步处理流程, 以及单端登录安全策略。
6. **多平台客户端开发**: 分别使用 Kotlin Jetpack Compose 和 Vue.js 3 开发了功能完整的 Android 和 Web 客户端。Android 端采用 Material 3 Expressive 薄荷色主题, Web 端采用 Aegis Slate 暗色 Material 3 设计系统。两个客户端均实现了普通消息与隐写消息的无缝切换。
7. **系统测试与验证**: 对系统各项功能进行了全面测试, 包括端到端加密正确性、双算法隐写嵌入提取、消息生命周期管理、跨平台兼容性等。

1.4 论文组织结构

本文共分为六章，各章内容安排如下：

第一章绪论：介绍了研究背景与意义，综述了国内外图像隐写术和端到端加密的研究现状，阐述了本文的主要研究内容和论文组织结构。

第二章相关技术与理论基础：介绍了图像隐写术的基本概念、可证安全隐写的理论基础、Pulsar 和 SparSamp 两种隐写算法的工作原理、端到端加密的密钥协商与加密封装机制，以及系统开发所涉及的关键技术栈。

第三章系统设计：从需求分析出发，详细阐述了系统的总体架构设计、服务端设计（API 接口、数据库、WebSocket 通信协议、离线消息队列、双算法隐写服务）、客户端设计（架构、本地存储、UI 导航）和安全设计（JWT 认证、密码安全、端到端加密、隐写密钥派生、单端登录）。

第四章系统实现：分别介绍了服务端、Android 客户端和 Web 客户端的具体实现过程，包括项目结构、关键模块的类与方法设计，以及端到端加密封装、隐写消息收发、消息状态管理等核心功能的实现细节。

第五章系统测试与分析：描述了系统的测试环境、功能测试（含端到端加密测试和双算法测试）、隐写性能分析和跨平台兼容性测试的过程与结果。

第六章总结与展望：总结了本文的主要工作和贡献，分析了系统的不足之处，并对未来的研究方向进行了展望。

2 相关技术与理论基础

本章介绍与系统设计和实现密切相关的理论基础和技术栈，包括图像隐写术的基本概念、可证安全隐写理论、Pulsar 和 SparSamp 两种隐写算法的工作原理、端到端加密机制以及系统开发涉及的关键技术。

2.1 图像隐写术概述

图像隐写术是指将秘密信息嵌入到数字图像中，使得含密图像 (Stego Image) 在视觉上与原始载体图像 (Cover Image) 无显著差异，从而实现隐蔽通信的技术^[1]。根据嵌入域的不同，图像隐写方法主要分为以下几类：

2.1.1 空间域隐写

空间域隐写直接修改图像像素值来嵌入信息。最经典的是 LSB (Least Significant Bit) 替换方法，将秘密信息的每一比特替换载体图像对应像素的最低有效位^[3]。该方法实现简单、嵌入容量大，但由于直接改变了像素的统计分布，容易被隐写分析检测。

为降低嵌入失真，研究者提出了多种自适应隐写方法。HUGO^[11] 通过最小化高阶残差统计量的变化来选择嵌入位置；WOW^[12] 利用小波变换获取像素权重，优先在纹理复杂区域嵌入；S-UNIWARD^[13] 在空间域和 JPEG 域统一使用方向滤波器组计算嵌入代价，是目前抗检测能力较强的方法之一。

2.1.2 变换域隐写

变换域隐写在图像经过某种数学变换（如离散余弦变换 DCT、离散小波变换 DWT）后的系数上嵌入信息。典型代表是 JPEG 域隐写，如 F5 算法^[14] 通过减小 DCT 系数的绝对值嵌入信息，nsF5^[15] 进一步避免了收缩问题，J-UNIWARD^[13] 则实现了最优的嵌入代价分配。

2.1.3 基于生成模型的隐写

与修改已有图像不同，基于生成模型的隐写方法直接在图像生成过程中嵌入信息，不需要已有的载体图像。这类方法利用深度生成模型（如 GAN、VAE、Flow、扩

散模型）采样过程中的随机性嵌入秘密信息，使得含密图像与模型正常输出的图像在分布上完全一致，从根本上消除了隐写痕迹^[7]。

2.2 可证安全隐写理论

2.2.1 信息论安全模型

可证安全隐写（Provably Secure Steganography）的理论基础源于 Cachin^[19] 和 Hopper 等人^[6] 的工作。其核心思想可以用信息论的语言形式化描述：

设 P_C 表示正常载体（Cover）的分布， P_S 表示含密载体（Stego）的分布。隐写系统的安全性通过两个分布之间的统计距离来度量。常用的度量指标为 KL 散度（Kullback-Leibler Divergence）：

$$D_{KL}(P_S \| P_C) = \sum_x P_S(x) \log \frac{P_S(x)}{P_C(x)} \quad (2.1)$$

2.2.2 Cachin 安全准则

Cachin^[19] 提出了基于 KL 散度的隐写安全性分级：

- **完美安全** (ϵ -secure, $\epsilon = 0$)：若 $D_{KL}(P_S \| P_C) = 0$ ，即含密载体与正常载体的分布完全一致，则任何检测器都无法以优于随机猜测的概率检测隐写行为。
- **近似安全** (ϵ -secure, $\epsilon > 0$)：若 $D_{KL}(P_S \| P_C) \leq \epsilon$ ，其中 ϵ 为一个很小的正数，则隐写系统具有近似安全性。

实现完美安全的关键在于：隐写编码过程不能对载体分布引入任何偏差。基于生成模型的隐写方案通过在模型的采样过程中嵌入信息，天然地满足了这一要求——因为含密图像也是由同一个生成模型产生的，其分布与正常生成图像完全一致。

2.3 Pulsar 算法原理

Pulsar^[9] 是一种基于扩散模型的可证安全图像隐写算法，由 Jois、Beck 和 Kaptchuk 于 2024 年提出，发表于 ACM CCS 2024。该算法通过控制逆向去噪过程中最后一步调度器的方差噪声来源实现隐写嵌入。在实际系统中，Pulsar 使用 DDIM（Denoising Diffusion Implicit Models）^[22] 采样器以 50 步完成图像生成，相比原始 DDPM 的 1000 步大幅降低了计算开销，同时保持了生成质量。

2.3.1 扩散模型基础

扩散模型^[23] 是一类通过逐步去噪过程生成数据的深度生成模型，近年来在图像生成领域取得了突破性进展^[24]，主要包含前向扩散过程和逆向去噪过程：

前向扩散过程：从真实数据 x_0 出发，逐步添加高斯噪声，经过 T 步后得到近似纯噪声 $x_T \sim \mathcal{N}(0, I)$ ：

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I) \quad (2.2)$$

其中 β_t 为预定义的噪声调度参数。

逆向去噪过程：训练一个神经网络 ϵ_θ 预测每一步的噪声，从纯噪声 x_T 开始逐步去噪恢复图像：

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I) \quad (2.3)$$

其中 μ_θ 由模型预测的噪声计算得出， σ_t 为方差系数。每步去噪都需要引入随机噪声 $z_t \sim \mathcal{N}(0, I)$ 。扩散模型的前向加噪与逆向去噪过程如图 2.1 所示。

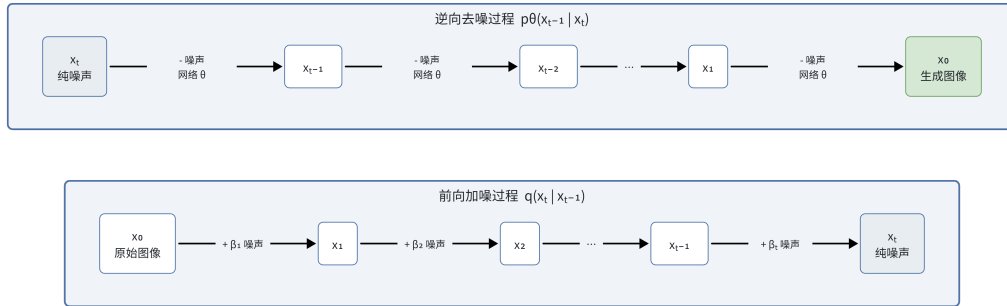


图 2.1 扩散模型前向加噪与逆向去噪过程

2.3.2 Pulsar 隐写编码原理

Pulsar 的核心思想是：利用扩散模型逆向去噪过程中最后一步调度器添加的方差噪声（variance noise）作为隐写信道。发送方和接收方共享密钥 K ，将其解析为三个 PRG 密钥：种子密钥 k_s 和两个参考密钥 k_0 、 k_1 。 k_s 用于同步前 $t-2$ 步去噪过程中的所有随机性，使双方生成完全相同的中间状态； k_0 和 k_1 则分别用于在最后一步调度器中采样不同的高斯噪声序列。

Pulsar 的嵌入过程分为离线阶段和在线阶段：

1. **离线阶段——模型同步**：使用 k_s 确定性地执行前 $t - 2$ 步逆向去噪，生成中间状态 s_{t-2} 。对该状态进行试编码，分别使用 k_0 和 k_1 生成两幅参考图像 img_0 和 img_1 ，将两幅图像之间的像素差异按幅值划分为 u 个桶 (bucket)，每个桶对应一个差异区域，根据各区域的实际误码率选择合适的纠错码参数。
2. **离线阶段——消息预编码**：使用选定的纠错码对秘密消息 m 进行编码，得到扩展后的比特序列 m_{ECC} 。
3. **在线阶段——噪声采样**：对 m_{ECC} 中的每一比特 $m_{\text{ECC}}[j]$ ，若其值为 0，则使用密钥 k_0 采样该像素位置的方差噪声 $r_{t-1}[j]$ ；若为 1，则使用密钥 k_1 采样。
4. **在线阶段——图像生成**：使用编码后的噪声向量 r_{t-1} 执行第 $t - 1$ 步调度器，再经确定性的最终步生成含密图像 img 。

上述过程的核心在于：每一比特消息通过选择不同的 PRG 密钥来控制对应像素的方差噪声来源。由于 k_0 和 k_1 产生的噪声均服从相同的高斯分布，含密图像的统计特性与正常生成图像不可区分，满足可证安全性。Pulsar 支持离线-在线分离，前 $t - 2$ 步的模型推理和区域估计可在消息未知时预先完成，实际编码时仅需执行最后两步调度器和纠错编码。

Pulsar 嵌入流程如图 2.2 所示。



图 2.2 Pulsar 隐写嵌入流程

2.3.3 Pulsar 消息提取

提取过程与嵌入过程对称：

1. **离线阶段**：接收方使用 k_s 执行前 $t - 2$ 步去噪，得到与发送方一致的中间状态 s_{t-2} 。然后分别使用 k_0 和 k_1 采样最后一步的方差噪声，生成参考图像 img_0 和 img_1 。
2. **在线阶段——比特恢复**：对含密图像 img 的每个编码像素 j ，计算 $|\text{img}[j] - \text{img}_0[j]|$ 与 $|\text{img}[j] - \text{img}_1[j]|$ 的大小，取距离更近的一方对应的比特值作为恢复值。

3. **消息解码**：对恢复的比特序列 m_{ECC} 执行纠错码的恢复算法，还原原始秘密消息 m 。

Pulsar 提取流程如图 2.3 所示。

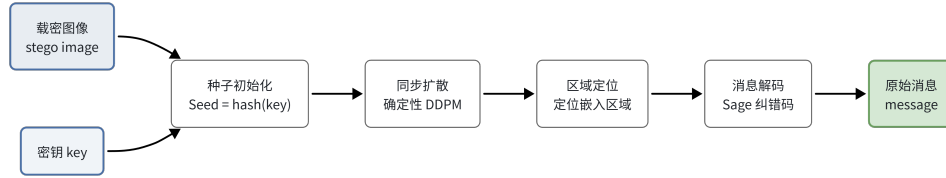


图 2.3 Pulsar 消息提取流程

2.3.4 纠错编码机制

Pulsar 的信道可建模为二进制对称信道 (BSC_p)，其误码率因图像区域而异。原论文采用二进制级联码 (concatenated code) 实现变码率纠错：外码为 Reed-Solomon 码，内码为通过手工搜索和经验测试确定的短二进制码，使其在给定 BSC_p 上的解码错误概率尽可能低且码率接近信道容量 $1 - H_2(p)$ 。

系统根据差异区域的误码率逐区域选择纠错码：从低误码率区域开始，为每个区域分配码率匹配的级联码；当某一区域的误码率过高、超出可用纠错码的纠错能力时，停止在该区域编码。这一策略充分利用了图像中高细节区域的低误码特性，避免了低细节背景区域的高误码拖累整体容量。实验表明，该方法在 256×256 像素图像上平均可嵌入 320–613 字节明文信息。

2.4 SparSamp 算法原理

SparSamp^[21] 是一种基于稀疏采样 (Sparse Sampling) 的可证安全隐写算法，其核心思想是通过消息驱动采样 (Message-Driven Sampling) 将秘密消息嵌入生成模型的采样过程中。与基于算术编码的方法不同，SparSamp 通过将消息段与伪随机数进行模加法结合，生成消息导出随机数 (Message-derived Random Number, MRN)，并利用 MRN 从概率分布中采样 token。当多个 MRN 采样到同一 token 导致解码歧义时，SparSamp 通过逐步缩小候选消息集合来增大相邻 MRN 之间的采样间隔，实现稀疏采样，从而消除歧义。该方法严格保持原始概率分布不变，且在每步采样中仅引入 $O(1)$ 的额外计算复杂度。本系统基于 IDDPM (Improved Denoising Diffusion

Probabilistic Models) [25] 的 P2-weighting 框架实现了 SparSamp 的图像隐写版本。

2.4.1 P2-weighting 框架

P2-weighting 是 IDDPM 的一种训练策略，通过为不同噪声水平分配不同的训练权重来提升生成质量。SparSamp 使用的 UNet 模型具有以下特征：

- **输出通道**：6 通道，其中 3 通道预测噪声 ϵ ，3 通道预测 v 插值参数。
- **模型架构**：基础通道数 128，通道倍增因子 (1, 1, 2, 2, 4, 4)，注意力分辨率 (16,)。
- **图像尺寸**：256 × 256 像素。
- **方差学习**：通过 v 参数化实现 σ 的学习。

2.4.2 SparSamp 嵌入流程

SparSamp 的嵌入过程基于消息驱动采样机制，主要分为以下步骤：

1. **密钥派生**：对共享密钥 K 计算 SHA-256 哈希，前 4 字节作为扩散采样种子，后 4 字节作为 SparSamp 编码种子。
2. **确定性扩散采样**：使用派生的种子设置所有随机数生成器（PyTorch、CUDA、NumPy），确保环境完全确定性。从纯噪声 x_T 出发，经 N 步（默认 10 步）逆向去噪到达 x_1 。
3. **最后一步分布计算**：在 $t = 0$ 时刻对 x_1 执行模型前向计算，得到每个像素的高斯分布参数 (μ_i, σ_i) ，并将连续高斯分布量化为 256 个离散 bin。
4. **消息驱动采样**：将消息打包为比特序列 (32 位长度头 + UTF-8 载荷)，按 l_m 比特一组划分为消息段。对每个像素，将当前消息段通过 bin2num 函数转换为 $[0, 1)$ 区间内的数值，与伪随机数 r_i 进行模加法得到 MRN: $r_i(m) = [r_i + \text{bin2num}(m)] \bmod 1$ 。使用 MRN 从该像素的量化分布中采样一个 bin 索引。
5. **稀疏采样消歧**：当多个候选消息段的 MRN 采样到同一个 bin 时，记录所有产生相同采样结果的候选 MRN，并在下一个像素的采样中使用这些候选 MRN 继续嵌入。随着候选集合 N_m 的缩小，相邻 MRN 之间的距离 $\frac{1}{N_m}$ 增大，采样变得更加稀疏，从而消除解码歧义，直到候选集合缩小为 1（即消息段被唯一确定）。
6. **图像生成**：采样完成后的 bin 索引数组 (shape 3 × 256 × 256) 转换为像素值，保存为 PNG 图像。

SparSamp 嵌入流程如图 2.4 所示。



图 2.4 SparSamp 嵌入流程

2.4.3 SparSamp 提取流程

提取过程是嵌入的对称操作：使用相同的密钥派生相同的种子，执行相同的扩散采样和分布计算，然后从像素值反向推导出 bin 索引。利用相同的伪随机数序列和 MRN 更新逻辑（sparse 函数），逐步缩小候选消息集合，直到 $N_m = 1$ 时即可唯一确定当前消息段的值。提取时先读取 32 位长度头确定消息总长度，再依次恢复各消息段并拼接为完整消息。

2.4.4 状态缓存机制

SparSamp 的扩散采样和分布计算耗时较长（约 50 秒），因此系统对每个（模型、密钥）对的计算结果进行缓存。首次请求时执行完整的扩散流程并缓存中间状态 (x_1, μ, σ) ，后续相同（模型、密钥）对的请求直接使用缓存，实现瞬时返回。

2.5 端到端加密机制

本系统实现了基于对称助记词的端到端加密方案，主要包括密钥派生、消息加密封装和隐写密钥协商三个环节。E2EE 密钥派生层次如图 2.5 所示。

2.5.1 用户密钥派生

每个用户持有一个助记词短语（任意字符串），通过 PBKDF2-SHA256 算法派生 256 位用户密钥：

$$\text{userKey} = \text{PBKDF2}(\text{phrase}, \text{salt}_s, 100000, 256) \quad (2.4)$$

其中 $\text{salt}_s = \text{"stego chat-seed-v1"}$ 为固定的种子盐值，迭代次数为 100,000 次。派生完成后，计算用户密钥的 SHA-256 指纹（取前 8 字节）用于配对验证：

$$\text{fingerprint} = \text{SHA-256}(\text{userKey})[0 : 8] \quad (2.5)$$

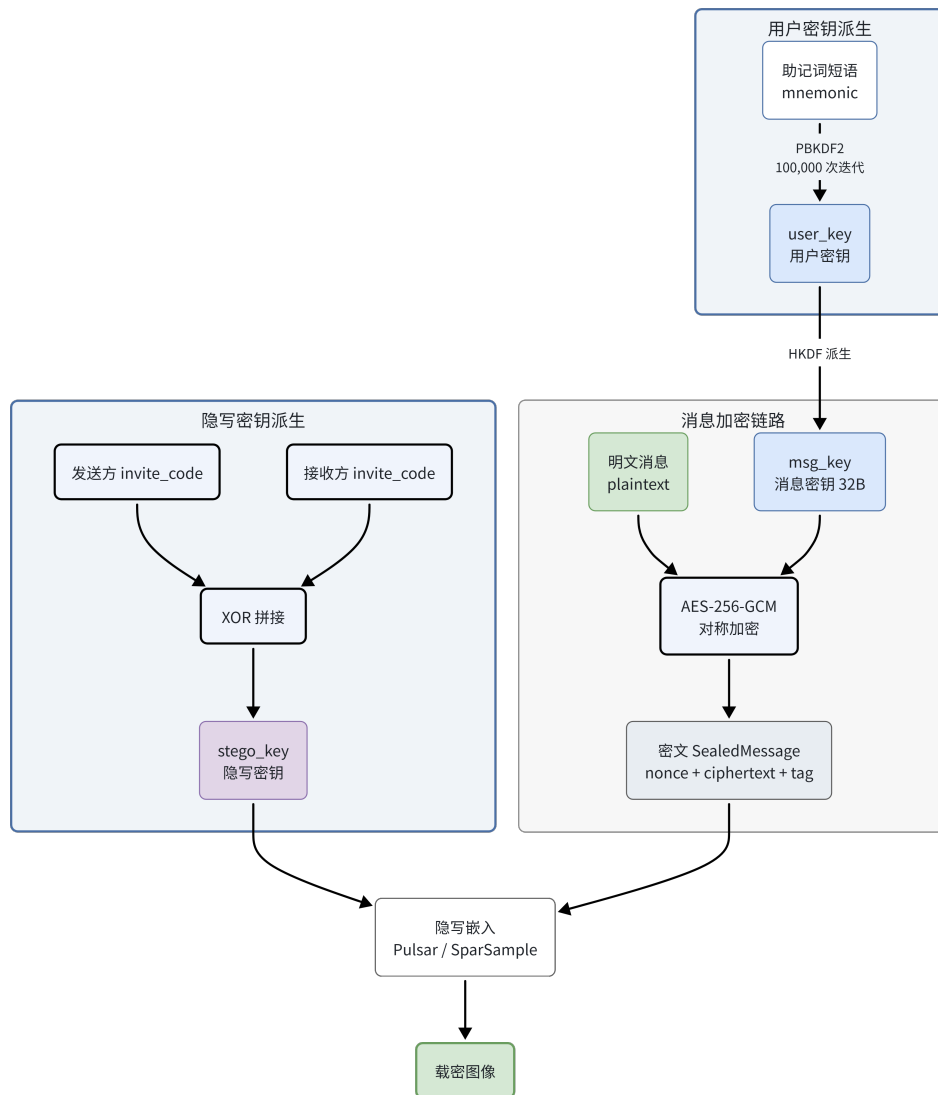


图 2.5 E2EE 密钥派生层次

指纹以冒号分隔的十六进制格式显示（如 a1:b2:c3:d4:e5:f6:a7:b8），用户可通过对比指纹验证密钥配对的正确性。

2.5.2 消息密钥协商

通信双方的用户密钥通过对称的 HKDF-SHA256 算法^[26] 协商出消息密钥。为保证对称性（即 $\text{deriveMkey}(A, B) = \text{deriveMkey}(B, A)$ ），双方的用户密钥按字典序排列后拼接：

$$\text{mkey} = \text{HKDF}(\text{concat}(\min(K_A, K_B), \max(K_A, K_B)), \text{salt}_m, \text{info}, 256) \quad (2.6)$$

其中 $\text{salt}_m = \text{"stegochat-mkey-v1"}$, $\text{info} = \text{"e2ee"}$ 。消息密钥仅在内存中存在, 不进行持久化存储, 每次切换聊天对象时重新计算。

2.5.3 消息加密封装

消息使用 AES-256-GCM 算法^[27] 加密, 封装为 SealedMessage 格式:

1. 生成 12 字节随机 nonce。
2. 使用消息密钥和 nonce 对明文执行 AES-256-GCM 加密, 得到密文和 128 位认证标签。
3. 构造 JSON 信封: $\{v: 1, n: \text{base64url}(\text{nonce}), c: \text{base64url}(\text{ciphertext} + \text{tag})\}$ 。
4. 将整个 JSON 字符串进行 base64url 编码, 得到最终的密封消息字符串。

解封时执行逆操作: base64url 解码、JSON 解析、验证版本号、AES-256-GCM 解密。解密失败时返回空值, 支持对未加密的旧格式消息的兼容处理。

2.5.4 隐写密钥协商

在隐写聊天模式下, 隐写算法需要一个共享密钥作为伪随机数生成器的种子。系统使用通信双方的邀请码进行 XOR 运算来生成隐写密钥:

$$K_{\text{stego}} = \text{inviteCode}_A \oplus \text{inviteCode}_B \quad (2.7)$$

XOR 运算具有对称性 ($A \oplus B = B \oplus A$), 因此双方无需协商即可独立计算出相同的隐写密钥。邀请码的长度差异通过循环 XOR 处理。生成的密钥以十六进制字符串形式传递给隐写算法的 API 接口。

2.5.5 隐写聊天的双重保护

隐写聊天模式下, 消息经历“先加密、再隐写”的双重保护流程:

1. 发送方使用消息密钥 (mkey) 对明文执行 AES-256-GCM 加密, 得到 SealedMessage。
2. 将 SealedMessage 字符串作为载荷, 使用隐写密钥 (K_{stego}) 调用隐写嵌入 API。

3. 隐写算法将加密后的载荷编码到扩散模型生成的图像中,返回含密图像的 base64 数据。
4. 含密图像通过 **WebSocket** 发送给接收方。
5. 接收方使用相同的隐写密钥调用隐写提取 API, 得到 **SealedMessage** 字符串。
6. 接收方使用消息密钥解封 **SealedMessage**, 还原明文消息。

加密-隐写双重保护的完整流程如图 2.6所示。

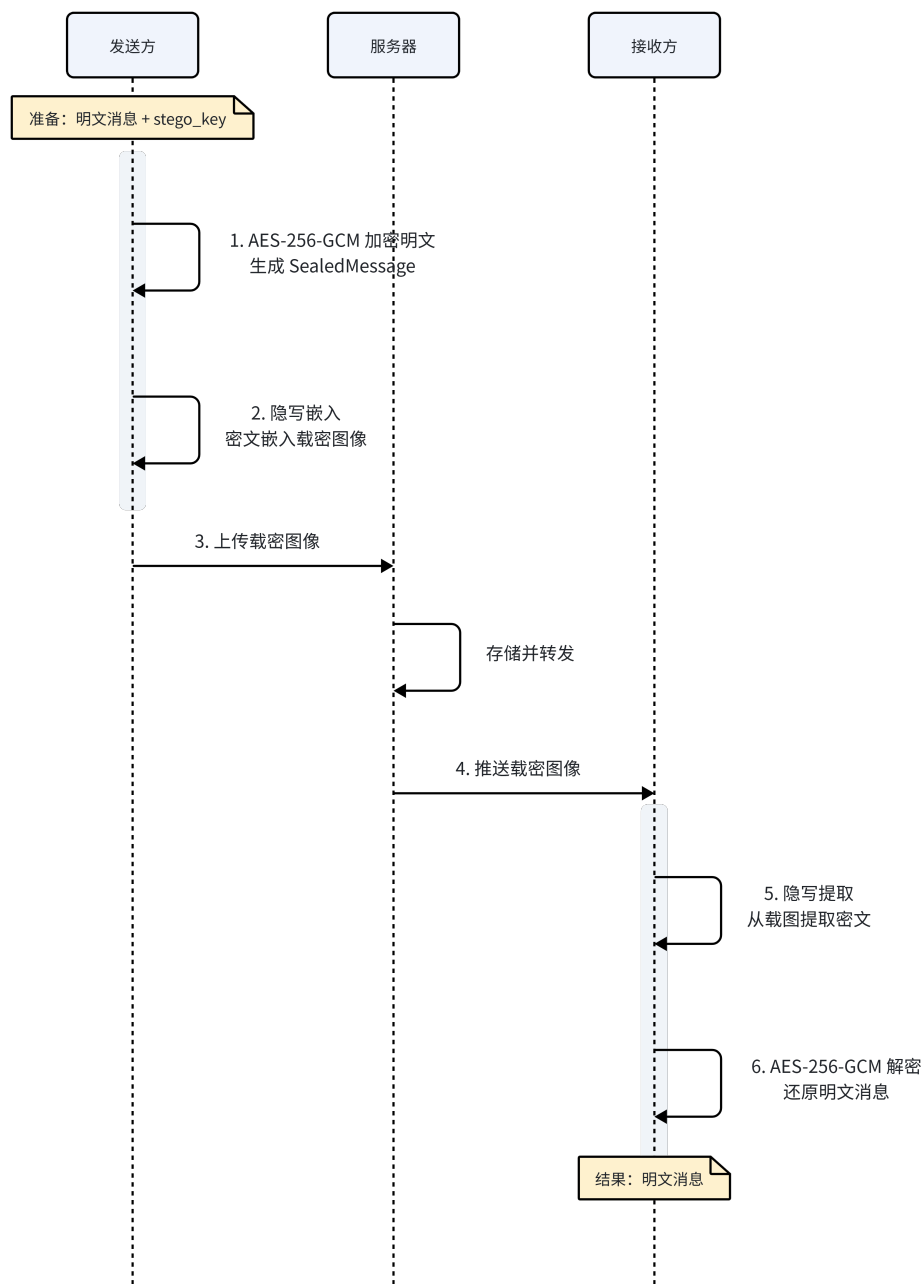


图 2.6 加密-隐写双重保护流程

由于隐写嵌入的载荷已经是加密后的密文，即使服务端或攻击者提取出了隐写内容，也无法解密得到原始消息。

2.6 关键技术栈

2.6.1 FastAPI 框架

FastAPI^[28] 是一个基于 Python 的现代 Web 框架，具有以下特点：

- 基于 ASGI（Asynchronous Server Gateway Interface）标准，原生支持异步编程。
- 自动生成 OpenAPI 文档和交互式 API 测试界面。
- 基于 Python 类型注解的请求参数验证和序列化。
- 高性能，与 Node.js 和 Go 等框架性能相当。

本系统使用 FastAPI 构建服务端 RESTful API 和 WebSocket 服务，使用 uvicorn 作为 ASGI 服务器。

2.6.2 WebSocket 协议

WebSocket 是一种在单个 TCP 连接上进行全双工通信的协议^[29]。与传统的 HTTP 请求-响应模式不同，WebSocket 允许服务端主动向客户端推送消息，适用于实时通信场景。本系统使用 WebSocket 实现即时消息推送、消息状态回执（已发送/已送达/已读）、消息撤回通知、在线状态感知和好友请求通知等功能。

2.6.3 Jetpack Compose

Jetpack Compose^[30] 是 Google 推出的 Android 现代声明式 UI 框架。与传统的 XML 布局相比，Compose 直接使用 Kotlin 代码描述 UI，支持响应式编程模型，代码更简洁，开发效率更高。本系统的 Android 客户端完全基于 Jetpack Compose 构建，采用 Material 3 Expressive 设计语言。

2.6.4 Vue.js 3

Vue.js 3^[31] 是一个渐进式 JavaScript 前端框架，采用组合式 API（Composition API）提供更灵活的逻辑复用能力。本系统使用 Vue.js 3 配合 TypeScript 开发 Web 客户端，使用 Pinia 进行状态管理，使用 Vue Router 管理页面路由，使用 Tailwind CSS

实现样式系统。

2.6.5 本地存储技术

为实现离线数据持久化，系统在不同平台采用了相应的本地存储方案：

- **SQLite**：服务端使用 SQLite 作为轻量级关系型数据库，通过 aiosqlite 库实现异步操作，启用 WAL 模式提升并发读写性能。
- **Room**：Android 客户端使用 Room 持久化库，它是 SQLite 的抽象层，提供编译时 SQL 验证和与 Kotlin 协程的无缝集成。
- **IndexedDB**：Web 客户端使用浏览器原生 IndexedDB 数据库存储聊天记录和用户数据，通过 idb 库简化操作。
- **DataStore**：Android 客户端使用 Jetpack DataStore 存储 JWT 令牌和用户信息，替代传统的 SharedPreferences。

2.6.6 密码学库

系统在不同平台使用相应的密码学实现：

- **服务端**：使用 Python 标准库 hashlib 实现 PBKDF2-HMAC-SHA256 密码哈希，使用 python-jose 实现 JWT 令牌的签发和验证。
- **Android 客户端**：使用 Java 标准库 javax.crypto 实现 AES-256-GCM 加密和 HKDF 密钥派生，使用 java.security 实现 PBKDF2。
- **Web 客户端**：使用 Web Crypto API (crypto.subtle) 实现所有密码学操作，包括 PBKDF2、HKDF 和 AES-GCM。

2.7 本章小结

本章介绍了图像隐写术的基本概念和分类，阐述了可证安全隐写的理论基础和 Cachin 安全准则，详细分析了 Pulsar 和 SparSamp 两种隐写算法的嵌入、提取和编码机制，介绍了端到端加密的密钥派生、消息加密封装和隐写密钥协商方案，并介绍了系统开发涉及的关键技术栈。这些理论和技术为后续的系统设计与实现奠定了基础。

3 系统设计

本章从需求分析出发，详细阐述系统的总体架构设计、服务端设计、客户端设计 and 安全设计方案。

3.1 需求分析

3.1.1 功能需求

通过对可证安全图像隐写通信系统的应用场景分析，确定系统的功能需求如下：

1. **用户管理**：支持用户注册、登录、注销，每个用户拥有唯一的邀请码标识和可选的端到端加密助记词配置。
2. **好友管理**：通过邀请码添加好友，支持好友请求的发送、接受、拒绝和屏蔽。
3. **实时聊天**：支持一对一文本消息的即时收发，具备完整的消息生命周期管理（已发送/已送达/已读回执/消息撤回），消息持久化存储于本地。
4. **端到端加密**：支持基于对称助记词的端到端加密，消息在客户端加密后传输，服务端无法获取明文。
5. **隐写消息**：在聊天过程中支持隐写模式，可将加密后的消息嵌入图像中发送，接收方可提取并解密查看隐藏的消息。
6. **独立隐写工具**：提供独立的消息嵌入和提取页面，支持用户手动输入密钥和选择算法进行隐写操作。
7. **双算法支持**：系统支持 Pulsar 和 SparSamp 两种隐写算法，用户可在独立工具中选择算法，聊天模式使用默认算法。
8. **跨平台支持**：系统需同时支持 Android 和 Web 两个平台，不同平台的用户可以互相通信。

3.1.2 非功能需求

1. **安全性**：隐写算法须满足可证安全条件；用户认证采用 JWT 令牌机制；密码存储使用安全的哈希算法（PBKDF2）；消息传输使用端到端加密。
2. **实时性**：消息收发延迟应在秒级以内（不含隐写计算时间）。
3. **可靠性**：当用户离线时，消息应缓存于服务端，待用户上线后自动投递；消息状态应准确反映实际传递情况。

4. **易用性**：客户端界面简洁直观，隐写功能与普通聊天无缝集成，端到端加密配置简单。
5. **可扩展性**：隐写算法应支持插件式注册，便于后续添加新的算法实现。

3.2 系统总体架构

系统采用客户端-服务端（C/S）架构，由服务端、Android 客户端和 Web 客户端三部分组成。系统总体架构如图 3.1 所示。

各组件的职责分工如下：

- **服务端**：负责用户认证与管理、好友关系维护、WebSocket 连接管理与消息路由、离线消息缓存、双算法隐写服务的执行（嵌入与提取计算）。服务端不接触消息明文——端到端加密在客户端完成。
- **Android 客户端**：提供移动端的用户界面，包括登录注册、联系人管理、聊天界面和隐写工具，使用 Room 数据库本地存储聊天记录，使用 DataStore 存储认证令牌。负责端到端加密的密钥管理和消息加解密。
- **Web 客户端**：提供浏览器端的用户界面，功能与 Android 客户端对齐，使用 IndexedDB 本地存储聊天记录。使用 Web Crypto API 实现密码学操作。

服务端与客户端之间通过两种通信方式交互：

- **RESTful API (HTTP)**：用于用户注册登录、隐写嵌入提取、邀请码查询、算法列表查询等请求-响应式操作。
- **WebSocket**：用于实时消息推送、消息状态回执（已发送/已送达/已读）、消息撤回通知、好友请求通知等需要双向实时通信的场景。

3.3 服务端设计

3.3.1 API 接口设计

服务端 API 按照功能模块划分为四组路由，如表 3.1 所示。

隐写相关的 API 接口均支持 `algorithm` 和 `model` 可选参数，用于指定使用哪种隐写算法和预训练模型。不指定时使用默认算法（SparSamp）和默认模型。

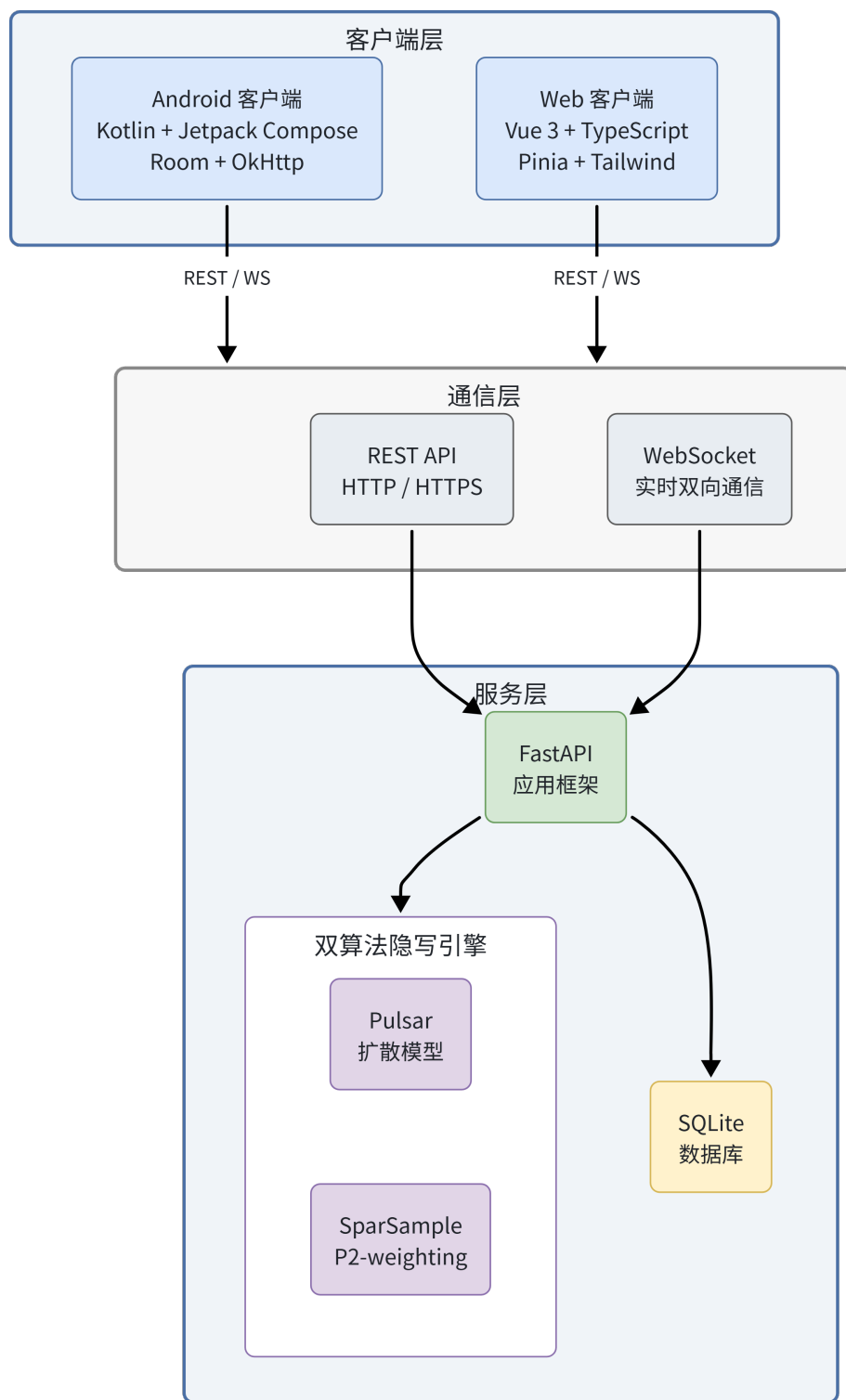


图 3.1 系统总体架构

表 3.1 服务端 API 接口设计

端点	方法	功能
/api/v1/auth/register	POST	用户注册
/api/v1/auth/login	POST	用户登录
/api/v1/auth/me	GET	获取当前用户信息
/api/v1/auth/logout	POST	用户登出
/api/v1/auth/user/{id}/public	GET	获取用户公开信息
/api/v1/invite/my-code	GET	获取自身邀请码
/api/v1/invite/reset	POST	重置邀请码
/api/v1/invite/{code}	GET	根据邀请码查找用户
/api/v1/invite/user-code/{user_id}	GET	获取指定用户邀请码
/api/v1/stego/algorithms	GET	获取可用算法及模型列表
/api/v1/stego/models	GET	获取指定算法的模型列表
/api/v1/stego/embed	POST	消息嵌入
/api/v1/stego/extract	POST	消息提取
/api/v1/stego/capacity	POST	容量检查
/api/v1/stego/max-capacity	GET	获取最大容量
/ws?token={jwt}	WebSocket	实时通信连接

3.3.2 数据库设计

服务端使用 SQLite 数据库存储用户数据。数据库包含三张核心表，如表 3.2 所示。SQLite 启用 WAL（Write-Ahead Logging）模式以提升并发读写性能。

message_queue 表在 expires_at 和 to_user_id 字段上建立索引，以优化过期清理和用户消息查询的性能。

好友关系不存储在服务端数据库中，而是通过客户端本地数据库维护，服务端仅负责转发好友请求消息。这种设计减轻了服务端的存储负担，同时保护了用户的社交关系隐私。

3.3.3 WebSocket 通信协议设计

WebSocket 连接建立后，客户端和服务端通过 JSON 格式的消息进行通信。消息类型设计如表 3.3 所示。

典型的 WebSocket 通信交互流程如图 3.2 所示。

chat 类型的消息携带以下字段：from_user_id（发送者 ID，服务端自动填充）、

表 3.2 服务端数据库表结构

表名	字段	类型	说明
users	id	TEXT (PK)	用户 UUID
	username	TEXT (UNIQUE)	用户名 (3-32 字符)
	password_hash	TEXT	密码哈希 (PBKDF2)
	created_at	DATETIME	创建时间
invite_codes	code	TEXT (PK)	邀请码 (8 位字母数字)
	user_id	TEXT (FK, UNIQUE)	所属用户
	created_at	DATETIME	创建时间
message_queue	id	TEXT (PK)	消息 UUID
	to_user_id	TEXT (FK)	目标用户
	payload	TEXT	消息 JSON 内容
	created_at	DATETIME	创建时间
	expires_at	DATETIME	过期时间 (默认 24 小时)

表 3.3 WebSocket 消息类型

消息类型	说明
chat	普通文本聊天消息 (含加密后的 SealedMessage)
ack	服务端确认收到消息, 返回消息 ID 和时间戳
delivered	消息已送达目标设备的回执
read	消息已被目标用户阅读的回执
revoke	消息撤回通知
typing	正在输入指示 (不持久化)
friend_request	好友请求
friend_accept	接受好友请求
friend_reject	拒绝好友请求
first_contact	首次联系通知 (自动触发)
kicked	单端登录踢出通知
auth_failed	认证失败通知

from_username (发送者用户名, 服务端自动填充)、to_user_id (接收者 ID)、content (消息内容, 为加密后的 SealedMessage 字符串)、content_type (消息类型: text 或 stego)、stego_image (隐写图像的 base64 数据, 仅 stego 类型)。

3.3.4 消息生命周期设计

系统实现了完整的消息生命周期管理，每条消息经历以下状态变迁：

1. **sending**：消息已创建，等待服务端确认。
2. **sent**：服务端返回 **ack**，确认已接收消息。
3. **delivered**：目标设备收到消息，返回 **delivered** 回执。
4. **read**：目标用户阅读消息，返回 **read** 回执。
5. **failed**：消息发送失败（连接断开等）。

此外，已发送的消息支持撤回操作（**revoke**），撤回后消息内容被清除，显示“消息已撤回”状态。撤回操作有时间限制（默认 120 秒）。消息状态变迁如图 3.3 所示。

3.3.5 离线消息队列设计

当目标用户不在线时，服务端将消息暂存于 **message_queue** 表中，并设置过期时间（默认 24 小时）。当用户重新连接 **WebSocket** 时，服务端自动查询并投递该用户的所有未过期离线消息，投递完成后从队列中删除。同时，服务端启动定时任务每 5 分钟清理过期消息。

3.3.6 双算法隐写服务设计

隐写服务采用统一的算法注册表模式，支持 **Pulsar** 和 **SparSamp** 两种算法并行运行。算法注册表维护每个算法的元信息（名称、模型列表、服务类），以及默认算法标识。

- **Pulsar 服务**：封装外部 **Pulsar Python** 包，支持 **CelebA-HQ**、**Church**、**Bedroom**、**Cat** 四种预训练 **DDPM** 模型。模型文件通过 **HuggingFace Hub** 或本地目录加载。
- **SparSamp 服务**：基于项目内实现的 **IDDPM P2-weighting** 框架，支持 **FFHQ**、**AFHQ Dog**、**Flower**、**CUB** 四种预训练模型。模型文件为 **.pt** 格式，存放在 **models/sparsample/** 目录下。

两种算法均通过 **run_in_threadpool** 在线程池中执行，避免阻塞异步事件循环。每个（算法、模型、密钥）组合对应一个独立的算法实例，通过字典缓存实现复用。

3.4 客户端设计

3.4.1 Android 客户端架构

Android 客户端采用 MVVM (Model-View-ViewModel) 架构模式，如图 3.4所示。

Android 客户端包含 36 个 Kotlin 源文件，按功能划分为以下包：

- **api/**: API 客户端 (Retrofit 接口定义和 DTO 数据类)。
- **crypto/**: 端到端加密工具类 (密钥派生、AES-GCM 加解密、SealedMessage 信封)。
- **data/local/**: 本地数据存储 (Room 数据库、DataStore、DAO)。
- **data/remote/**: WebSocket 客户端。
- **ui/navigation/**: 导航图定义。
- **ui/screens/**: 各页面的 Composable 函数。
- **ui/theme/**: Material 3 主题定义 (颜色、字体、形状)。
- **ui/viewmodel/**: ViewModel 层。

底部导航栏提供四个入口：消息（会话列表）、联系人、隐写工具和我的（个人设置）。

3.4.2 Web 客户端架构

Web 客户端采用 Vue.js 3 组合式 API 开发，使用 Pinia 进行集中式状态管理。

- **视图层**: Vue 单文件组件 (SFC)，共 10 个页面组件和 3 个公共组件。
- **状态管理层**: Pinia Store 管理全局状态，包括 AuthStore (认证状态)、ChatStore (聊天与 WebSocket 连接、E2EE 消息密钥管理)、ContactStore (联系人管理)、SettingsStore (自身 E2EE 配置)。
- **数据持久层**: 使用 IndexedDB (通过 idb 库封装) 存储聊天消息、联系人数据、黑名单和 E2EE 配置。
- **API 层**: 使用 Axios 发送 HTTP 请求，自动注入 JWT 令牌。
- **WebSocket 层**: 使用浏览器原生 WebSocket API，封装为 WsClient 类，支持指数退避重连。
- **密码学层**: 使用 Web Crypto API 实现 E2EE 密钥派生和消息加解密。

Web 客户端采用 Aegis Slate 暗色 Material 3 设计系统，主色调为深海军蓝(#0b1326)，

强调色为蓝色（#adc6ff），使用 Tailwind CSS 实现响应式布局。侧边导航栏宽 64px，提供会话、联系人、工具和设置四个导航入口。Web 客户端架构如图 3.5 所示。

3.4.3 本地存储设计

Android 客户端的 Room 数据库（版本 2）包含以下实体：

- **ContactEntity**：联系人信息（用户 ID、用户名、状态、添加时间、对方助记词、对方用户密钥十六进制、对方指纹十六进制）。
- **MessageEntity**：聊天消息（消息 ID、联系人 ID、方向、内容、内容类型、隐写图像、状态、撤回标记、创建时间）。
- **BlacklistEntity**：黑名单（用户 ID、用户名、屏蔽时间）。
- **UserSettingsEntity**：当前用户的 E2EE 配置（助记词、用户密钥十六进制、指纹十六进制），单行表。

Web 客户端的 IndexedDB 数据库（数据库名 stego-app，版本 2）包含四个对象存储：contacts、messages、blacklist、settings，结构与 Android 端对应。

3.5 安全设计

3.5.1 威胁模型

本系统的安全设计基于以下威胁模型和信任假设：

- **攻击者能力**：假设攻击者能够被动监听服务端的所有通信流量，包括 WebSocket 消息和 HTTP 请求；能够获取服务端数据库的完整内容（含用户密码哈希、邀请码和离线消息队列）；能够截获并读取客户端与服务端之间的所有网络数据包。
- **安全目标**：即使攻击者具备上述能力，也无法获取用户聊天消息的明文内容（消息机密性）；无法伪造合法用户的消息（消息完整性与认证性）；无法确定含密图像中是否隐藏了秘密信息（隐写不可检测性）。
- **信任假设**：用户信任自己所使用的客户端应用（Android 或 Web）未被篡改；助记词短语通过安全的带外渠道（如面对面）在通信双方之间共享；隐写算法本身的可证安全性基于底层扩散模型的 KL 散度保证。

需要注意的是，当前系统的隐写密钥通过邀请码 XOR 派生，而邀请码存储在服务端数据库中，因此隐写密钥的安全性依赖于服务端的可信度。这是系统在安全性方面的一个已知局限，详见小节 6.2 的讨论。

3.5.2 JWT 认证机制

系统使用 JSON Web Token (JWT) 进行用户身份认证。用户登录成功后，服务端签发包含用户 ID 和用户名的 JWT 令牌，有效期为 7 天。客户端在后续的 HTTP 请求中通过 Authorization 头部携带令牌，WebSocket 连接通过 URL 查询参数传递令牌。服务端使用 HMAC-SHA256 (HS256) 算法验证令牌的完整性和有效性。

JWT 密钥通过三级降级策略生成和持久化：优先读取环境变量 `JWT_SECRET`，其次读取项目目录下的 `.jwt_secret` 文件，最后随机生成并写入文件（权限 0600），确保服务重启后已有令牌不会失效。

3.5.3 密码安全

用户密码使用 PBKDF2-HMAC-SHA256 算法进行哈希存储，迭代次数为 100,000 次，每个用户使用随机生成的 UUID4 作为盐值。迭代次数的选择基于实际部署环境的权衡：OWASP 2023 年推荐的最低迭代次数为 600,000 次，但本系统的服务端运行在普通开发硬件上，100,000 次迭代已能在保证安全强度的同时维持登录响应在 200ms 以内。对于生产环境部署，建议将迭代次数提升至 600,000 次以上。哈希结果以十六进制编码，存储格式为 盐值 \$ 哈希值。验证时使用 `hmac.compare_digest` 进行时间安全比较，防止时序攻击。

3.5.4 端到端加密设计

端到端加密的密钥管理遵循以下原则：

- **助记词本地存储**：用户的助记词短语和派生的用户密钥仅存储在客户端本地数据库中，不传输到服务端。
- **消息密钥内存驻留**：对称消息密钥（mkey）仅在 ViewModel 的内存变量中存在，不进行持久化存储，每次切换聊天对象时重新计算。
- **密文传输**：所有聊天消息在客户端加密为 SealedMessage 格式后传输，服务端只看到密文。
- **指纹验证**：用户可通过对比对方的密钥指纹（8 字节 SHA-256 摘要）来验证密钥配对的正确性。

3.5.5 隐写密钥派生

在隐写聊天模式下，系统自动使用通信双方的邀请码进行 XOR 运算生成隐写密钥。XOR 运算具有对称性，双方无需协商即可独立计算出相同的密钥。生成的密钥以十六进制字符串形式传递给隐写 API，作为伪随机数生成器的种子。

3.5.6 单端登录策略

为防止多设备同时登录引发的数据一致性问题，系统实施单端登录策略。当用户在新设备登录时，服务端向旧连接发送 `kicked` 消息通知其被踢出，旧客户端收到通知后清除本地令牌并跳转到登录页面。5 秒后服务端强制关闭旧连接作为兜底措施。主动登出清除所有本地数据，而被踢出仅清除令牌，保留聊天记录便于用户重新登录后查看。

3.6 本章小结

本章从功能需求和非功能需求出发，设计了系统的总体架构，详细阐述了服务端的 API 接口、数据库、WebSocket 通信协议（含完整的消息生命周期）、离线消息队列和双算法隐写服务的设计方案，以及客户端的架构和本地存储设计。同时，从 JWT 认证、密码安全、端到端加密、隐写密钥派生和单端登录五个方面介绍了系统的安全设计。

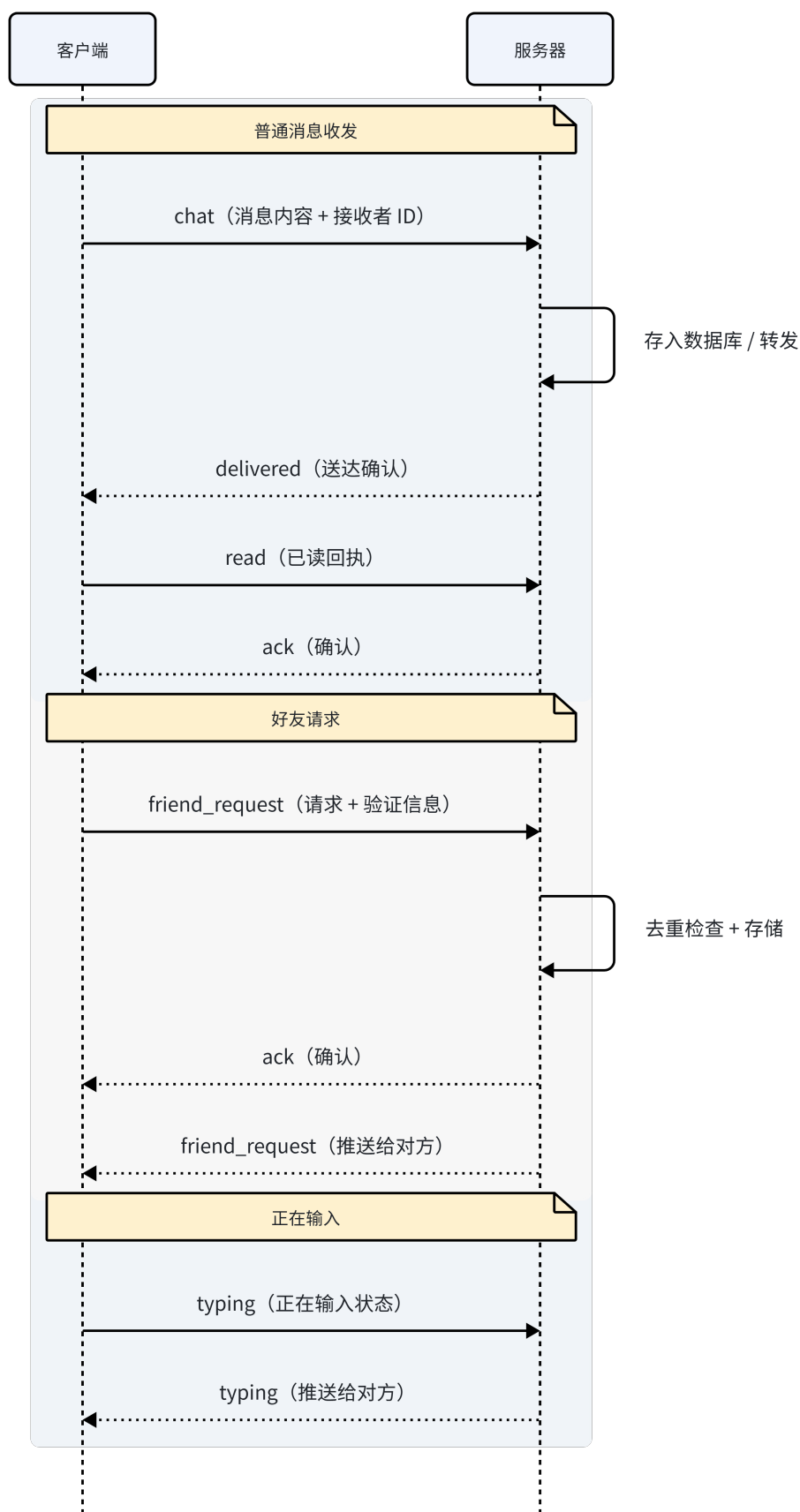


图 3.2 WebSocket 通信时序

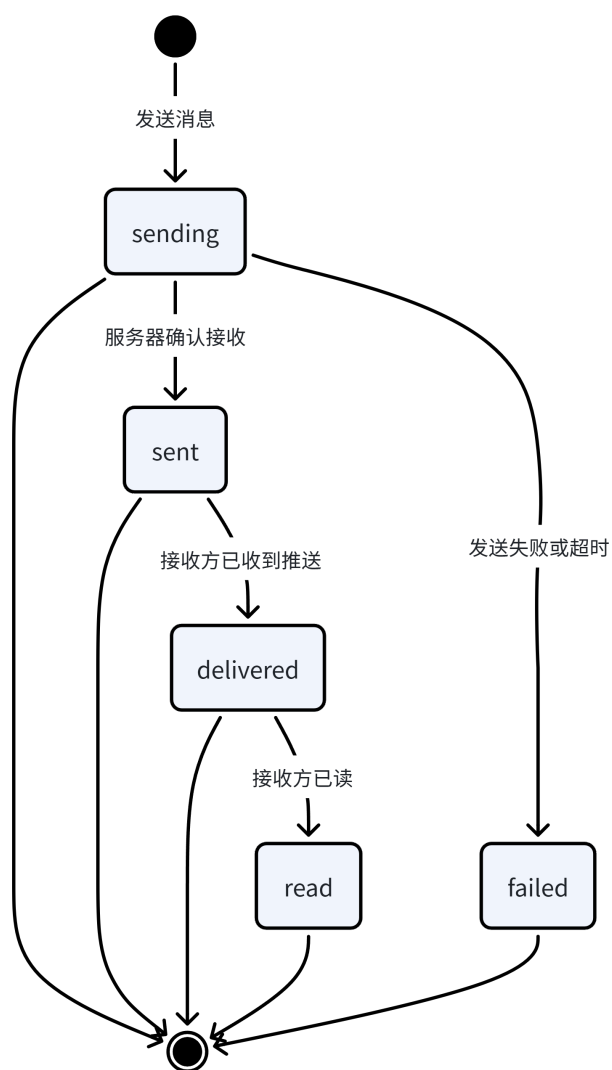


图 3.3 消息状态机

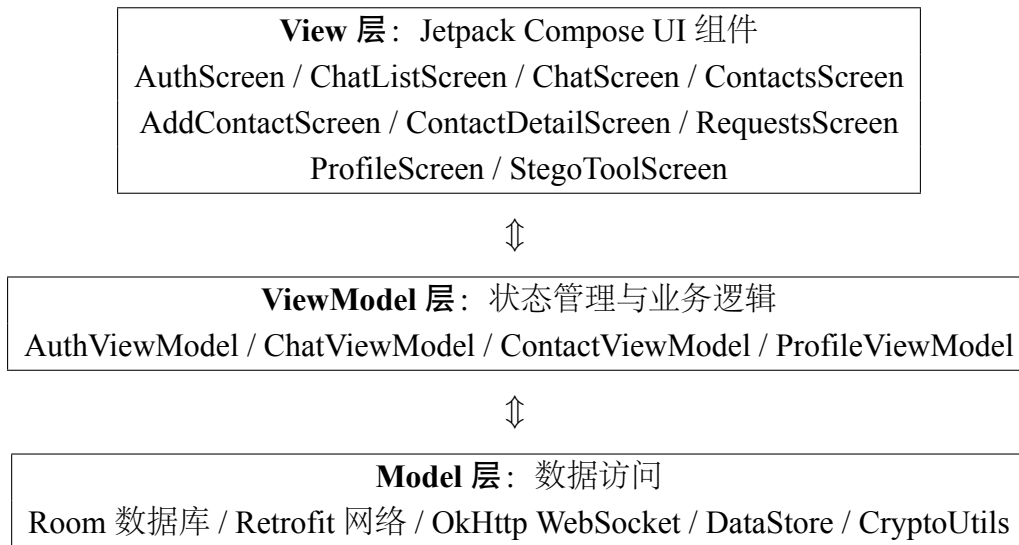


图 3.4 Android 客户端 MVVM 架构

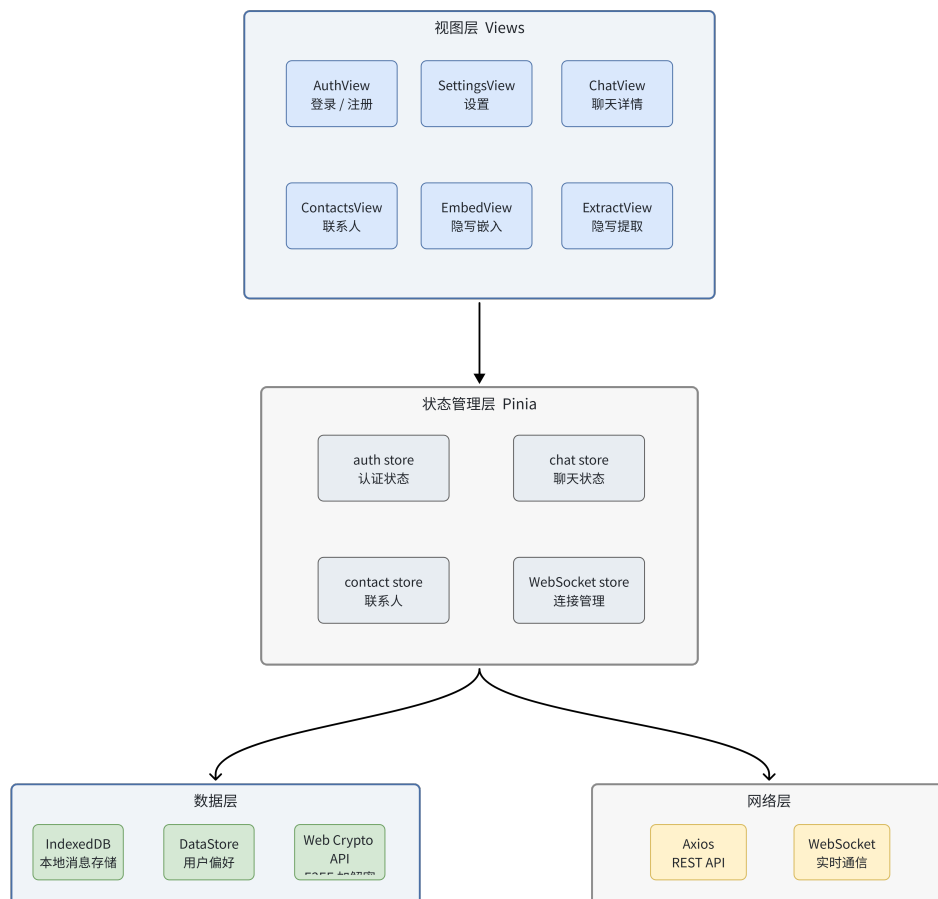


图 3.5 Web 客户端架构

4 系统实现

本章分别介绍服务端、Android 客户端和 Web 客户端的具体实现过程，以及关键功能的实现细节。

4.1 服务端实现

4.1.1 FastAPI 应用结构

服务端基于 FastAPI 框架构建，采用模块化的项目结构。主要目录组织如下：

```
server/
+-- main.py           # 应用入口、生命周期管理
+-- config.py         # 配置项（JWT、数据库、队列等）
+-- database.py       # SQLite 数据库初始化与连接
+-- dependencies.py   # FastAPI 依赖注入（认证）
+-- api/v1/
|   +-- auth.py       # 认证路由（注册/登录/登出）
|   +-- invite.py     # 邀请码路由
|   +-- stego.py       # 隐写路由（双算法统一入口）
|   `-- ws.py         # WebSocket 路由
+-- services/
|   +-- auth_service.py # 密码哈希、JWT、用户 CRUD
|   +-- pulsar_service.py # Pulsar 算法服务封装
|   +-- sparsample_service.py # SparSamp 算法服务封装
|   +-- ws_manager.py  # WebSocket 连接管理
|   `-- queue_service.py # 离线消息队列
+-- services/sparsample/
|   +-- core.py        # 嵌入/提取/扩散采样/高斯量化
|   `-- guided_diffusion/ # UNet 模型定义
+-- sage/             # SageMath 纠错码实现
`-- models/           # 数据模型
```

应用启动时通过 FastAPI 的生命周期管理（lifespan）完成数据库初始化和定时任务注册。路由通过 `include_router` 方法注册到主应用实例。CORS 中间件允许跨域请求以支持 Web 客户端访问。

4.1.2 双算法隐写服务集成

系统通过算法注册表模式实现 Pulsar 和 SparSamp 的统一管理。注册表结构维护每个算法的 ID、显示名称、模型列表和服务类引用。聊天模式的默认算法为 SparSamp（`CHAT_DEFAULT_ALGORITHM = "sparsample"`）。

PulsarService 采用类方法设计，主要包含：

- **实例管理**: 使用字典缓存 Pulsar 实例, 以 " 模型 ID: 密钥十六进制 " 组合作为缓存键。首次请求时自动加载预训练模型并执行区域估计。
- **嵌入操作**: 接收消息字节、模型 ID 和密钥, 调用 Pulsar 的 `generate_with_regions` 方法生成含密图像, 将图像保存为 PNG 格式并返回字节数据。
- **提取操作**: 接收含密图像字节、模型 ID 和密钥, 调用 `reveal_with_regions` 方法提取隐藏消息。

SparSampService 同样采用类方法设计, 但内部实现更为复杂:

- **状态缓存**: 对每个 (模型 ID、密钥) 对, 首次请求时执行完整的 IDDPM 扩散采样流程, 缓存中间状态 (x_1 、 μ 、 σ 、最大容量)。后续请求直接使用缓存, 实现瞬时返回。
- **嵌入操作**: 将消息打包为比特序列, 通过消息驱动采样机制逐像素嵌入——将消息段与伪随机数模加结合生成 MRN, 使用 MRN 从量化分布中采样 bin 索引, 并通过稀疏采样机制消除解码歧义, 最终生成 256×256 的 PNG 图像。
- **提取操作**: 使用相同的密钥和种子恢复扩散状态, 从像素值反向推导 bin 索引, 利用相同的 MRN 更新逻辑逐步缩小候选消息集合, 直到唯一确定消息段。

两种算法均使用 `run_in_threadpool` 在线程池中执行 CPU 密集型操作, 避免阻塞异步事件循环。

4.1.3 WebSocket 连接管理

`ConnectionManager` 类负责管理所有活跃的 WebSocket 连接, 使用字典维护用户 ID 到 WebSocket 实例的映射关系。核心实现包括:

- **连接建立**: 验证 JWT 令牌后建立连接。若该用户已有活跃连接 (多设备登录), 先向旧连接发送 `kicked` 消息, 并设置 5 秒延迟强制关闭。
- **消息路由**: 根据消息中的目标用户 ID 查找对应的 WebSocket 连接, 若目标用户在线则直接转发, 否则将消息加入离线队列。
- **消息生命周期**: 服务端为每条 chat 消息生成唯一 ID 和时间戳, 向发送者返回 `ack` 确认, 支持 `delivered`、`read` 和 `revoke` 状态的转发。
- **离线消息投递**: 用户连接建立后, 自动查询并投递该用户的所有缓存消息。

4.1.4 离线消息队列

离线消息使用 SQLite 的 `message_queue` 表实现持久化存储。消息入队时设置 24 小时过期时间，服务端通过 `asyncio.create_task` 启动定时清理任务，每 5 分钟删除过期消息。用户重新上线时，系统自动投递所有未过期消息并从队列中移除。

4.2 Android 客户端实现

4.2.1 界面实现

Android 客户端使用 Jetpack Compose 构建声明式 UI，采用 Material 3 Expressive 设计语言，主色调为薄荷色（#4ECDC4），使用 Inter 字体族。主要包含以下页面：

- **AuthScreen**：登录和注册界面，通过 `PrimaryTabRow` 实现标签切换。品牌标识为 EnhancedEncryption 图标 + “StegoCrypt” 文字。支持表单验证（用户名 3 字符以上，密码 6 字符以上）。
- **ChatListScreen**：会话列表，显示所有已接受的联系人，按最后消息时间排序。每行显示头像、显示名称、最后消息预览（文本或“隐写图像”图标）和时间戳。
- **ChatScreen**：聊天界面，顶部显示 E2EE 状态横幅（三种状态：自身未配置/对方未配置/已加密），中间为消息列表，底部为输入栏。支持普通消息和隐写消息的发送与接收。
- **ContactsScreen**：联系人列表，顶部显示好友请求入口（带未读计数），每行显示头像、名称和加密状态标签。
- **AddContactScreen**：通过邀请码添加联系人，支持搜索预览和备注输入。
- **ContactDetailScreen**：联系人详情页，支持配置对方的 E2EE 助记词、查看指纹、发起聊天和删除联系人。
- **RequestsScreen**：好友请求列表，支持接受、忽略和屏蔽三种操作。接受时自动处理队列中的暂存消息。
- **ProfileScreen**：个人设置页，包含邀请码管理（查看/复制/重置）和自身 E2EE 助记词配置（输入/保存/重置/指纹显示）。
- **StegoToolScreen**：独立隐写工具，通过 `SingleChoiceSegmentedButtonRow` 切换嵌入和提取两种模式，支持算法和模型选择。

页面导航使用 Jetpack Navigation Compose 实现，底部导航栏提供消息、联系人、

隐写工具和我的四个入口。

4.2.2 Room 数据库实现

使用 Room 持久化库定义本地数据库（版本 2），包含以下 DAO 操作：

- **ContactDao**：联系人的增删改查，支持按用户 ID 查询和全量查询，支持更新 E2EE 对方密钥字段。
- **MessageDao**：聊天消息的插入、按联系人 ID 查询（按时间排序）和状态更新。
- **BlacklistDao**：黑名单的增删查。
- **UserSettingsDao**：当前用户 E2EE 配置的读写（单行表，ID 固定为 0），支持 Flow 观察。

数据库版本 1 到版本 2 的迁移添加了联系人表的 E2EE 字段（peerPhrase、peerUserKeyHex、peerFingerprintHex）和 user_settings 表。

4.2.3 端到端加密实现

本节介绍第3节章中端到端加密设计的具体代码实现。CryptoUtils.kt 封装了所有密码学操作，包含以下常量和方法：

- **常量**：SEED_SALT（"stegochat-seed-v1"）、MKEY_SALT（"stegochat-mkey-v1"）、MKEY_INFO（"e2ee"）、PBKDF2_ITER（100,000）。
- **deriveUserKey(phrase)**：使用 PBKDF2WithHmacSHA256 从助记词派生 256 位用户密钥。
- **deriveMkey(selfKey, peerKey)**：按字典序排列两个用户密钥，拼接后使用 HKDF（HMAC-SHA-256）派生 32 字节消息密钥。
- **fingerprint(userKey)**：计算用户密钥的 SHA-256，取前 8 字节作为指纹。
- **SealedMessage.seal(mkey, plaintext)**：生成 12 字节随机 nonce，执行 AES-256-GCM 加密，构造 JSON 信封 {v:1,n:...,c:...}，base64url 编码。
- **SealedMessage.tryOpen(mkey, sealed)**：base64url 解码、JSON 解析、AES-256-GCM 解密。失败返回 null。

消息密钥（mkey）在 ChatViewModel 中通过 combine 操作符自动重算：当活跃联系人 ID、用户自身 E2EE 配置或联系人列表发生变化时，自动为当前聊天对象派生新的 mkey。

4.2.4 WebSocket 客户端实现

WsClient 类使用 OkHttp 库实现 WebSocket 客户端：

- URL 构造：将 HTTP(S) 地址替换为 WS(S) 地址，附加 ?token=<jwt> 查询参数。
- 消息解析：接收到的 JSON 文本解析为 WsMessage 对象，通过 SharedFlow 分发给 ViewModel。
- 重连机制：指数退避策略，最大间隔 30 秒，最多重连 10 次。
- 踢出检测：WebSocket 关闭码 4001 或 HTTP 401/403 触发踢出流程。

ChatViewModel 在 handleWsMessage 方法中按消息类型分发处理：

- **chat**：检查黑名单、判断是否为新联系人（好友请求场景）、执行 E2EE 解密、保存到本地数据库、发送 delivered 回执。
- **ack**：更新消息状态为”sent”。
- **delivered**：更新消息状态为”delivered”。
- **read**：更新消息状态为”read”。
- **revoke**：标记消息为已撤回，清除内容。
- **kicked**：断开连接，发射踢出事件。

4.2.5 隐写功能实现

聊天界面中的隐写模式通过一个切换开关控制，启用条件为邀请码已加载且 E2EE 已配置：

1. 用户切换到隐写模式后，界面显示容量指示器和 XOR 密钥信息。
2. 发送时先调用 SealedMessage.seal 对明文加密，再将 SealedMessage 字符串作为载荷调用 /api/v1/stego/embed 接口。密钥由双方邀请码 XOR 生成。
3. 嵌入成功后，将含密图像的 base64 数据作为 stego 类型消息通过 WebSocket 发送给对方。
4. 接收方收到隐写消息后，在聊天界面显示图像预览。长按可选择”提取秘密消息”，调用 /api/v1/stego/extract 接口提取载荷，再调用 SealedMessage.tryOpen 解密还原明文。

独立隐写工具（StegoToolScreen）不涉及 E2EE，用户手动输入消息和密钥，支持选择算法和模型。

4.3 Web 客户端实现

4.3.1 Vue 组件结构

Web 客户端的主要页面组件如表 4.1 所示。

表 4.1 Web 客户端页面组件

组件	路由	功能
AuthView	/login, /register	登录注册（标签切换）
ChatListView	/chats	会话列表
ChatView	/chat/:id	单个聊天（含 E2EE 状态）
ContactsView	/contacts	联系人列表
AddContactView	/contacts/add	添加联系人
ContactDetailView	/contacts/:id	联系人详情（E2EE 配置）
RequestsView	/contacts/requests	好友请求列表
ProfileView	/profile	个人设置（邀请码 + E2EE）
EmbedView	/embed	独立隐写嵌入工具
ExtractView	/extract	独立隐写提取工具

公共组件包括：SideNav（侧边导航栏，64px 宽，4 个导航入口）、MessageBubble（消息气泡，支持文本和隐写图像，右键菜单提供提取和保存操作）、MessageInput（输入栏，支持隐写模式切换和容量指示）。

4.3.2 Pinia 状态管理

使用 Pinia 管理全局状态，定义了四个 Store：

- **AuthStore**：管理用户认证状态。令牌和用户信息持久化到 localStorage。verify 方法在应用启动时验证令牌有效性。logout 清除所有本地数据，onKicked 仅清除令牌。
- **ChatStore**：管理 WebSocket 连接和消息收发。维护消息密钥（mkey，内存驻留）、邀请码、活跃联系人等状态。实现了完整的消息生命周期处理（ack/delivered/read/revoke）。隐写密钥通过双方邀请码 XOR 计算。
- **ContactStore**：管理联系人列表和黑名单。支持通过邀请码添加联系人、配置对方 E2EE 助记词、屏蔽和取消屏蔽用户。
- **SettingsStore**：管理当前用户的 E2EE 配置（助记词、用户密钥、指纹），持久化

到 IndexedDB 的 `settings` 存储。应用启动时通过 `hydrate` 方法恢复状态。

4.3.3 IndexedDB 本地存储

使用 `idb` 库封装 IndexedDB 操作，数据库名 `stego-app`，版本 2，包含四个对象存储：

- **contacts**: 联系人记录，主键 `user_id`，包含 E2EE 对方密钥字段。
- **messages**: 聊天消息，主键 `id`，按 `contact_id` 和 `created_at` 建立索引。
- **blacklist**: 黑名单，主键 `user_id`。
- **settings**: 应用设置，主键 `key`，存储 E2EE 自身配置。

4.3.4 端到端加密实现

Web 端的 E2EE 实现位于 `src/crypto/e2ee.ts`，使用 Web Crypto API 实现所有密码学操作。关键常量（`SEED_SALT`、`MKEY_SALT`、`MKEY_INFO`、`PBKDF2_ITER`）与 Android 端和 Python 服务端保持一致。

密钥派生流程与 Android 端完全对称：`deriveUserKey` 使用 `crypto.subtle.deriveBits` 实现 PBKDF2，`deriveMkey` 使用 `crypto.subtle.deriveBits` 实现 HKDF。`SealedMessage` 的 JSON 信封格式和 `base64url` 编码规则与 Android 端完全一致，确保跨平台消息互通。

`base64url` 编码采用 URL 安全变体（`-` 替代 `+`，`_` 替代 `/`），去除填充符 `=`，与 Android 端的 `java.util.Base64.getUrlEncoder().withoutPadding()` 对齐。

4.3.5 WebSocket 与隐写功能

Web 客户端的 WebSocket 实现使用浏览器原生 WebSocket API，封装为 `WsClient` 单例类。连接 URL 从 API 基础地址推导（HTTP 替换为 WS），支持指数退避重连（最大间隔 30 秒，最多 10 次）。

隐写消息收发流程与 Android 端一致：发送时先加密（`sealMessage`）再嵌入（`stegoApi.embed`），接收时先提取（`stegoApi.extract`）再解密（`openStegoPayload`）。隐写图像在 Web 端以纯 `base64` 字符串存储（不含 `data:` 前缀），渲染时动态添加前缀。

4.4 关键功能实现细节

4.4.1 单端登录机制

单端登录涉及服务端和客户端的协同处理：

1. **服务端**：ConnectionManager.connect 方法在新连接到来时，检查该用户是否已有活跃连接。若有，先发送 kicked 消息，然后创建异步任务在 5 秒后强制关闭旧连接（关闭码 4001）。
2. **Android 客户端**：WsClient 监听关闭码 4001，通过 SharedFlow 发射踢出事件。ChatViewModel 收到后断开 WebSocket，通知 UI 层显示提示并跳转登录页。
3. **Web 客户端**：WsClient 监听关闭码 4001，触发 kicked 事件。ChatStore 断开连接后通过 DOM 自定义事件通知 App.vue 执行页面跳转。

主动登出清除所有本地数据（令牌、聊天记录、联系人、E2EE 配置），而被踢出仅清除令牌，保留聊天记录和 E2EE 配置。单端登录的时序交互如图 4.1 所示。

4.4.2 好友请求流程

好友添加采用基于邀请码的异步请求机制：

1. 用户 A 输入用户 B 的邀请码，客户端调用 /api/v1/invite/{code} 查询对应用户信息。
2. 确认后，客户端在本地数据库添加联系人（状态为"accepted"），并通过 WebSocket 向用户 B 发送 first_contact 类型的 chat 消息。
3. 用户 B 收到消息时，若发送者不在联系人列表中，自动创建待处理请求。用户 B 可选择接受、忽略或屏蔽。
4. 接受时，联系人添加到本地数据库，队列中的暂存消息被处理。

需要处理的边界情况包括：重复请求的去重（通过检查联系人是否已存在）、离线请求的暂存与恢复、接受请求时恢复所有暂存的聊天消息等。好友请求流程如图 4.2 所示。

4.4.3 隐写消息收发流程

隐写消息的完整收发流程如下：

1. **发送方**：用户切换到隐写模式，输入文本消息。客户端获取双方邀请码，XOR

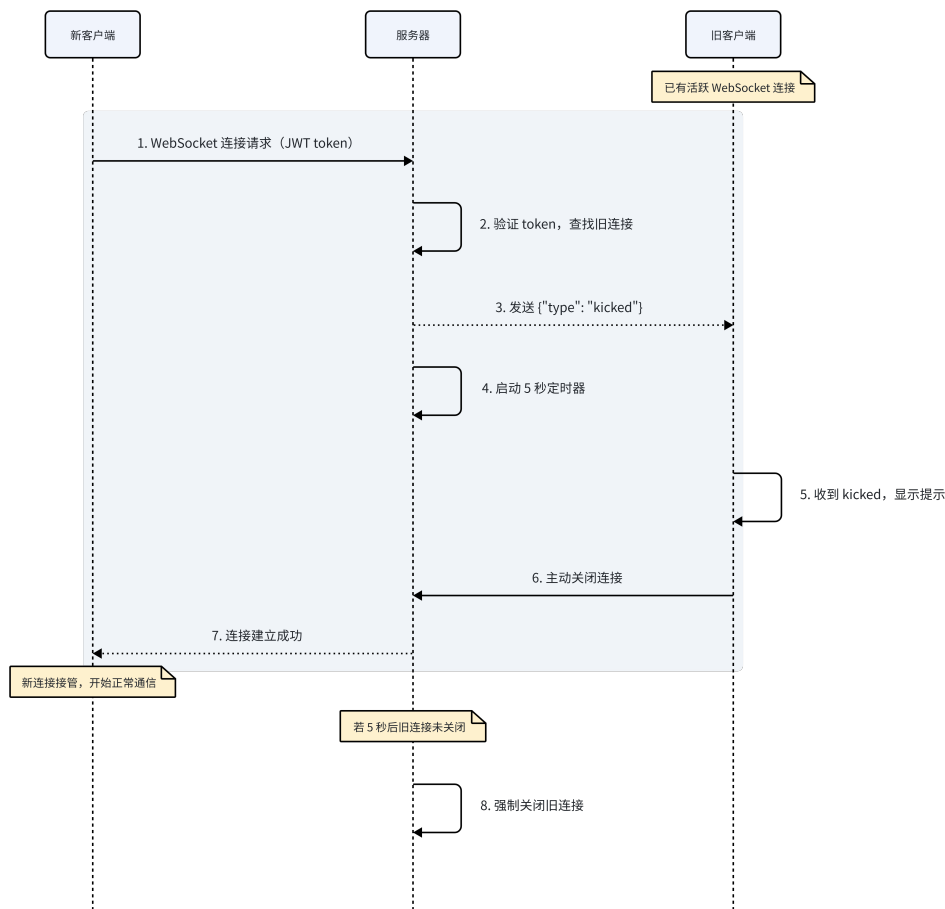


图 4.1 单端登录时序

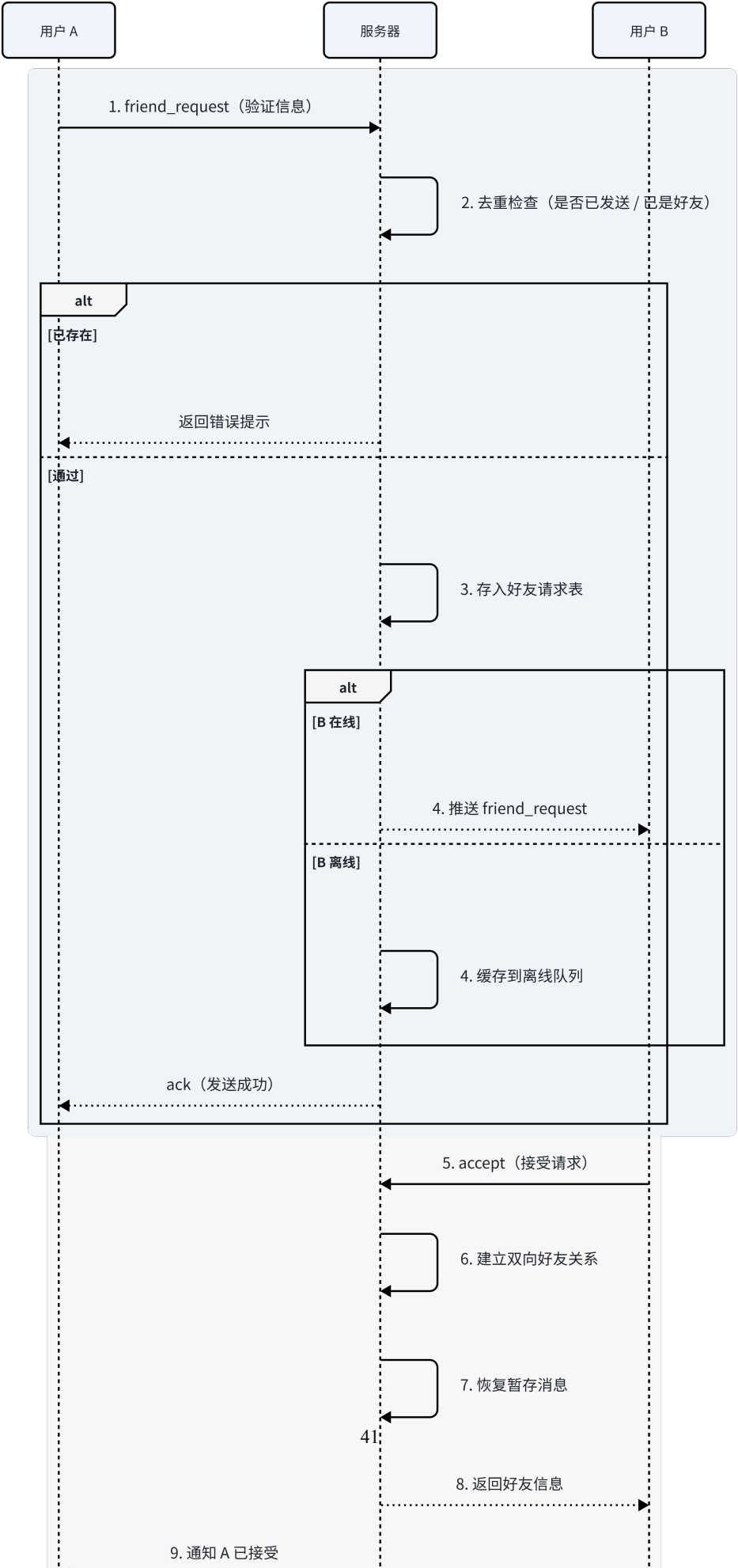
计算隐写密钥。先调用 `SealedMessage.seal` 加密明文，再将密封字符串作为载荷调用嵌入 API。API 返回含密图像的 base64 数据后，通过 WebSocket 以 `stego` 类型发送。

2. **服务端**：接收 WebSocket 消息，根据目标用户 ID 转发（或入离线队列）。服务端不解析 `stego_image` 的内容。
3. **接收方**：收到 `stego` 消息后，在聊天界面显示图像预览。用户选择提取，客户端使用相同的隐写密钥调用提取 API，得到密封字符串后调用 `SealedMessage.tryOpen` 解密还原明文。

隐写消息的完整收发时序如图 4.3 所示。

4.5 本章小结

本章详细介绍了系统三个组件的具体实现过程。服务端通过模块化的 **FastAPI** 应用集成了双算法隐写引擎、**WebSocket** 通信（含完整消息生命周期）和离线消息队列；**Android** 客户端基于 **MVVM** 架构使用 **Jetpack Compose** 构建了包含 9 个页面的完整 UI 和数据管理层；**Web** 客户端使用 **Vue.js 3**、**Pinia** 和 **Tailwind CSS** 实现了与 **Android** 端功能对齐的浏览器应用。同时，本章重点阐述了端到端加密的跨平台实现、双算法隐写服务的统一接口设计、单端登录、好友请求和隐写消息收发等关键功能的实现细节。



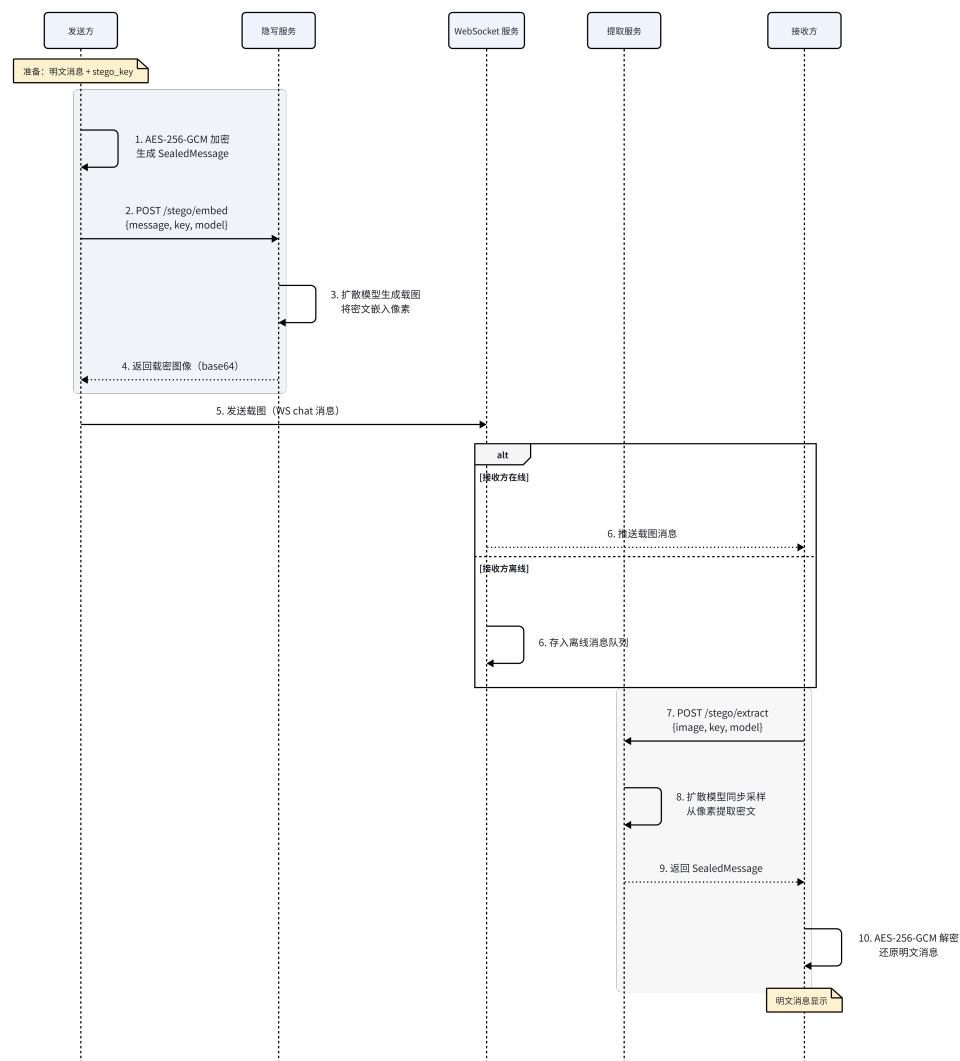


图 4.3 隐写消息收发时序

5 系统测试与分析

本章介绍系统的测试环境、功能测试、隐写性能分析以及跨平台兼容性测试的过程与结果。

5.1 测试环境

系统测试所使用的软硬件环境如表 5.1所示。

表 5.1 测试环境配置

项目	配置
操作系统	Ubuntu 22.04 LTS (WSL2)
CPU	Intel Core i7-12700H（笔记本）
GPU	NVIDIA GeForce RTX 3060 Laptop（6 GB，CUDA 12.x）
内存	16 GB
Python 版本	3.10+（Sage 环境）
Android 测试设备	Android 12+ 物理设备/模拟器
Web 浏览器	Google Chrome 120+
Node.js 版本	18+

服务端运行在本地开发环境中，使用 `uvicorn` 作为 ASGI 服务器，监听 8000 端口。`Pulsar` 和 `SparSamp` 算法在 Sage 数学软件环境中运行，使用 CUDA 加速扩散模型的推理计算。

5.2 功能测试

5.2.1 用户注册与登录测试

测试了用户注册和登录的正常流程与异常处理，测试用例及结果如表 5.2所示。
用户注册和登录后的界面效果如图 5.1所示。

5.2.2 好友管理测试

测试了好友添加的完整流程，包括邀请码查询、好友请求发送与处理等，测试用例及结果如表 5.3所示。

好友管理的界面效果如图 5.2所示。

表 5.2 用户注册与登录测试用例

编号	测试内容	结果
T-01	正常注册新用户	通过
T-02	注册已存在的用户名	通过（正确提示错误）
T-03	注册时用户名格式校验（过短/特殊字符）	通过
T-04	注册时密码长度校验（少于 6 字符）	通过
T-05	正确用户名密码登录	通过
T-06	错误密码登录	通过（正确提示错误）
T-07	登录后获取用户信息	通过
T-08	注册后自动生成邀请码	通过
T-09	登录令牌有效性验证（/auth/me）	通过
T-10	令牌过期后的请求拒绝	通过（401 响应）

表 5.3 好友管理测试用例

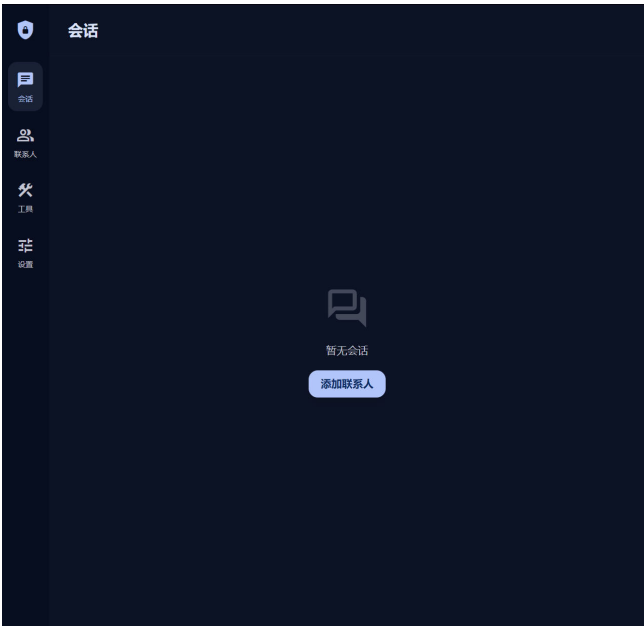
编号	测试内容	结果
T-11	通过邀请码查找用户	通过
T-12	发送好友请求（对方在线）	通过
T-13	发送好友请求（对方离线）	通过（离线缓存生效）
T-14	接受好友请求	通过
T-15	拒绝好友请求	通过
T-16	屏蔽用户	通过（后续消息被过滤）
T-17	重复发送好友请求（去重）	通过
T-18	重置邀请码	通过（旧码失效）
T-19	好友请求计数显示	通过



(a) Web 端注册页面



(b) Android 端注册页面



(c) 登录后主页

图 5.1 用户注册与登录界面



图 5.2 好友管理界面

5.2.3 消息收发与生命周期测试

测试了普通消息和隐写消息在不同场景下的收发功能，以及消息状态管理，测试用例及结果如表 5.4所示。

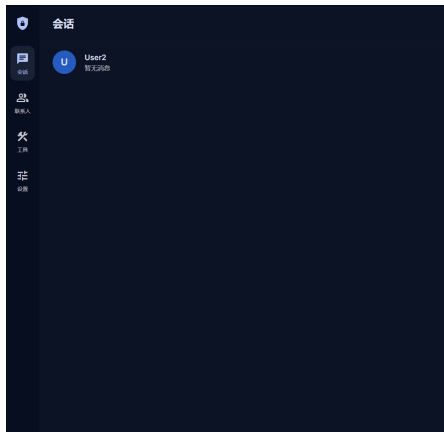
表 5.4 消息收发与生命周期测试用例

编号	测试内容	结果
T-20	发送普通文本消息（双方在线）	通过
T-21	发送普通文本消息（对方离线后上线）	通过
T-22	消息状态：sending → sent（回执）	通过
T-23	消息状态：sent → delivered（回执）	通过
T-24	消息状态：delivered → read（回执）	通过
T-25	发送隐写消息（嵌入与传输）	通过
T-26	接收隐写消息并提取文本	通过
T-27	消息本地持久化存储	通过
T-28	隐写图像保存到本地	通过

消息收发的界面效果如图 5.3所示。

5.2.4 端到端加密测试

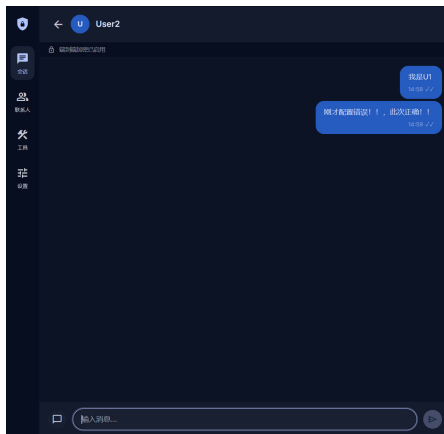
测试了端到端加密的密钥派生、消息加解密和跨平台兼容性，测试用例及结果如表 5.5所示。



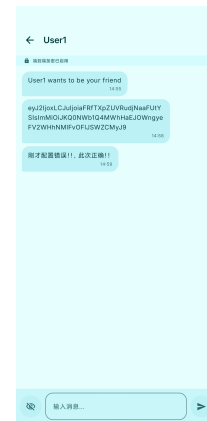
(a) Web 端聊天列表



(b) Android 端聊天列表



(c) Web 端普通消息



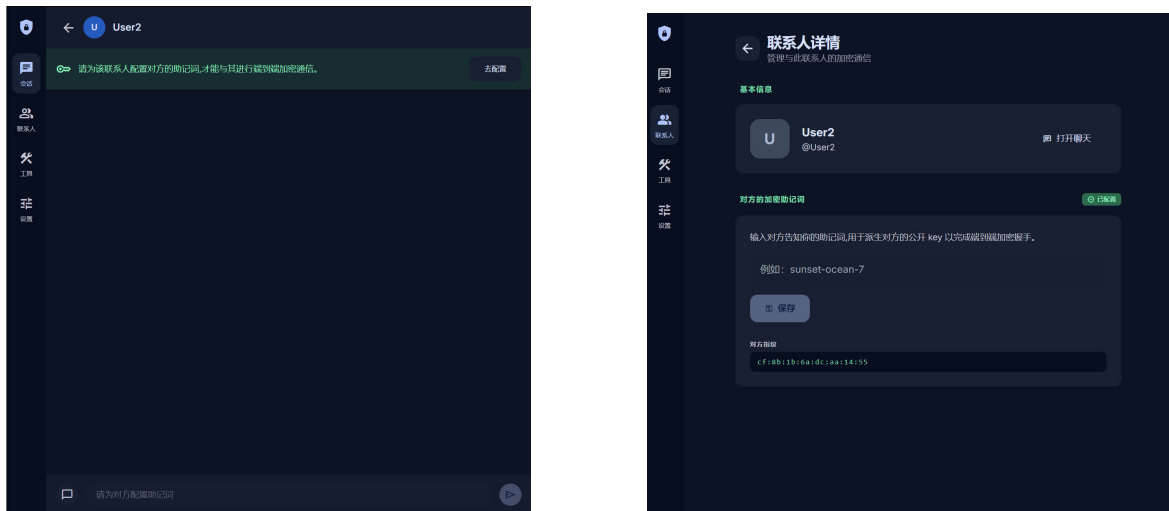
(d) Android 端普通消息

图 5.3 消息收发界面

表 5.5 端到端加密测试用例

编号	测试内容	结果
T-29	助记词保存与指纹生成	通过
T-30	用户密钥派生一致性（相同助记词）	通过
T-31	消息密钥对称性（ $\text{deriveMkey}(A,B) = \text{deriveMkey}(B,A)$ ）	通过
T-32	消息加密与解密（SealedMessage round-trip）	通过
T-33	错误密钥解密失败（返回 null）	通过
T-34	未加密消息兼容处理（legacy plaintext）	通过
T-35	E2EE 状态横幅显示（三种状态）	通过
T-36	指纹对比验证（双方一致）	通过
T-37	Android 加密 → Web 解密（跨平台）	通过
T-38	Web 加密 → Android 解密（跨平台）	通过

端到端加密的界面效果如图 5.4 所示。



(a) E2EE 状态横幅

(b) 指纹对比验证

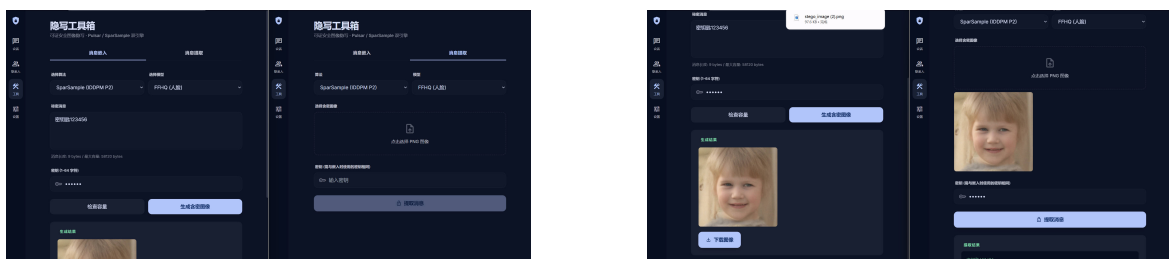
图 5.4 端到端加密功能

端到端加密使用固定的黄金测试向量（Golden Test Vectors）进行验证，确保 Android（Kotlin）、Web（TypeScript）和未来可能的服务端（Python）实现之间的互操作性。测试向量包括：已知助记词对应的用户密钥和指纹、已知密钥对和 **nonce** 对应的加密结果、SealedMessage 序列化格式等。

5.2.5 隐写嵌入与提取测试

测试了独立隐写工具页面的嵌入与提取功能，以及双算法切换，测试用例及结果如表 5.6 所示。

隐写嵌入和提取页面的测试效果如图 5.5 所示。



(a) 隐写嵌入页面

(b) 隐写提取页面

图 5.5 隐写嵌入与提取功能

表 5.6 隐写嵌入与提取测试用例

编号	测试内容	结果
T-39	Pulsar 算法：正常嵌入短文本消息	通过
T-40	Pulsar 算法：嵌入后正确提取消息	通过
T-41	SparSamp 算法：正常嵌入短文本消息	通过
T-42	SparSamp 算法：嵌入后正确提取消息	通过
T-43	算法切换：从 Pulsar 切换到 SparSamp	通过
T-44	模型切换：选择不同预训练模型	通过
T-45	超出容量限制时提示错误	通过
T-46	使用错误密钥提取（应失败）	通过
T-47	容量查询接口测试	通过
T-48	算法列表查询接口测试	通过
T-49	SparSamp 状态缓存（第二次请求瞬时返回）	通过

5.3 隐写性能分析

为客观评估两种隐写算法的实际性能，设计了对比实验，使用 5 个随机生成的 24 字节密钥和 3 种不同长度的消息（6、62、150 字节），直接调用算法服务层进行嵌入与提取测试。测试环境为 Ubuntu 22.04 LTS，配备 NVIDIA GPU 和 CUDA 加速。

5.3.1 算法性能对比

两种算法的性能对比如表 5.7 所示。

表 5.7 Pulsar 与 SparSamp 算法性能对比

指标	Pulsar (DDIM)	SparSamp (IDDPM P2)
平均嵌入容量 (bytes)	403	58112
平均嵌入耗时 (s)	1.9	12.4
平均提取耗时 (s)	2.4	0.03
提取正确率	100%	100%
跨密钥安全	是	是
输出图像格式	16-bit PNG	8-bit PNG
图像分辨率	256×256	256×256
可证安全性	是	是

表中 Pulsar 的数据基于 CelebA-HQ 256×256 预训练 DDIM 模型，SparSamp 的数据基于 FFHQ 256×256 预训练 IDDPM P2-weighting 模型。

从表 5.7可以看出两种算法在性能特征上存在显著差异：

1. **嵌入容量：**SparSamp 的平均嵌入容量（58,112 字节）远大于 Pulsar（403 字节），约为后者的 144 倍。这是因为 SparSamp 基于稀疏采样机制，通过消息驱动采样在每个像素的量化分布中嵌入消息段，编码空间与图像像素数和消息段长度成正比；而 Pulsar 基于二进制级联纠错码（外码 Reed-Solomon），编码容量受限于图像差异区域的误码率分布和可用纠错码的纠错能力。
2. **嵌入耗时：**Pulsar 的嵌入速度（平均 1.9 秒）明显快于 SparSamp（平均 12.4 秒）。Pulsar 仅需执行 50 步 DDIM 采样即可生成含密图像，而 SparSamp 需要执行 1000 步扩散采样（虽然 respaced 为 10 步，但每步计算量较大）。
3. **提取耗时：**SparSamp 的提取速度（平均 0.03 秒）远快于 Pulsar（平均 2.4 秒）。SparSamp 的提取过程仅需根据已知的采样结果和 PRNG 状态进行 MRN 更新与候选集合缩小，不涉及模型推理；而 Pulsar 的提取需要重新执行扩散采样以重建编码区域，再进行纠错码解码。
4. **正确性与安全性：**两种算法在所有 30 组往返测试中均实现 100% 提取正确率，跨密钥测试中均无法使用错误密钥提取原始消息，验证了两种算法的正确性和密钥安全性。

两种算法的性能对比如图 5.6所示。

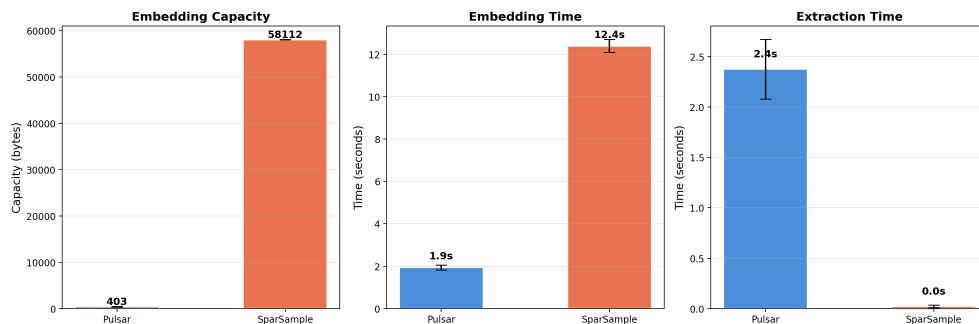


图 5.6 Pulsar 与 SparSamp 性能对比

综合来看，Pulsar 适合对嵌入速度和提取延迟敏感的应用场景，而 SparSamp 更适合需要大容量隐写的场景。在隐写聊天模式下，由于嵌入的载荷是 E2EE 加密后的 SealedMessage（包含 JSON 信封和 base64url 编码开销），系统将服务器返回的最大容量乘以 0.65 作为安全预算，以容纳加密带来的约 37% 的数据膨胀。

5.3.2 安全性分析

Pulsar 和 SparSamp 两种算法的可证安全性均已在各自的研究工作中得到理论证明。本次实验的跨密钥测试也从实践层面验证了这一特性：使用密钥 A 嵌入的含密图像，使用密钥 B 进行提取时无法恢复原始消息。其安全性保证基于以下共同原理：

1. **分布一致性**：含密图像与正常生成图像都由同一扩散模型生成，差异仅在于采样噪声的来源（PRG vs 真随机数），在分布不可区分的意义下，含密图像与正常生成图像不可区分（KL 散度趋近于 0）。
2. **密钥安全性**：不知道密钥的攻击者无法确定编码参数和噪声分配，因此无法从图像中恢复秘密信息。

在端到端加密层面，即使攻击者从隐写图像中提取出了载荷内容，由于载荷是 AES-256-GCM 加密后的密文，没有消息密钥（mkey）也无法解密。消息密钥的派生依赖于双方各自的助记词短语，助记词仅存储在客户端本地，不经过网络传输。

5.3.3 图像质量

两种算法生成的图像质量均取决于底层扩散模型的生成能力。由于信息嵌入仅改变了采样路径或最后一步的像素值分布，含密图像的视觉质量与正常生成图像无差异。使用 256×256 分辨率的预训练模型时，生成的图像具有较高的真实感。

为量化评估含密图像的质量损失，分别计算了含密图像与对应原始扩散模型生成图像之间的峰值信噪比（PSNR）、结构相似性（SSIM）以及弗雷歇起始距离（FID），结果如表 5.8 所示。其中 PSNR 和 SSIM 基于 20 组含密-原图配对计算均值，FID 基于 20 张含密图像与 20 张原始生成图像的分布距离计算。实验在 NVIDIA RTX 3060 Laptop GPU 上完成，总耗时约 47 分钟。

表 5.8 含密图像与原始生成图像的质量对比

算法 / 模型	PSNR (dB)	SSIM	FID
Pulsar / CelebA-HQ DDIM	60.04	1.0000	0.06
SparSamp / FFHQ IDDPM P2	42.77	0.9988	1.76

在实际聊天场景中使用隐写模式的效果如图 5.8 所示。发送方在隐写模式下输入文本消息后，系统自动将消息嵌入到扩散模型生成的含密图像中并发送给接收方；接收方点击图像即可提取出原始文本消息，整个过程对用户透明。

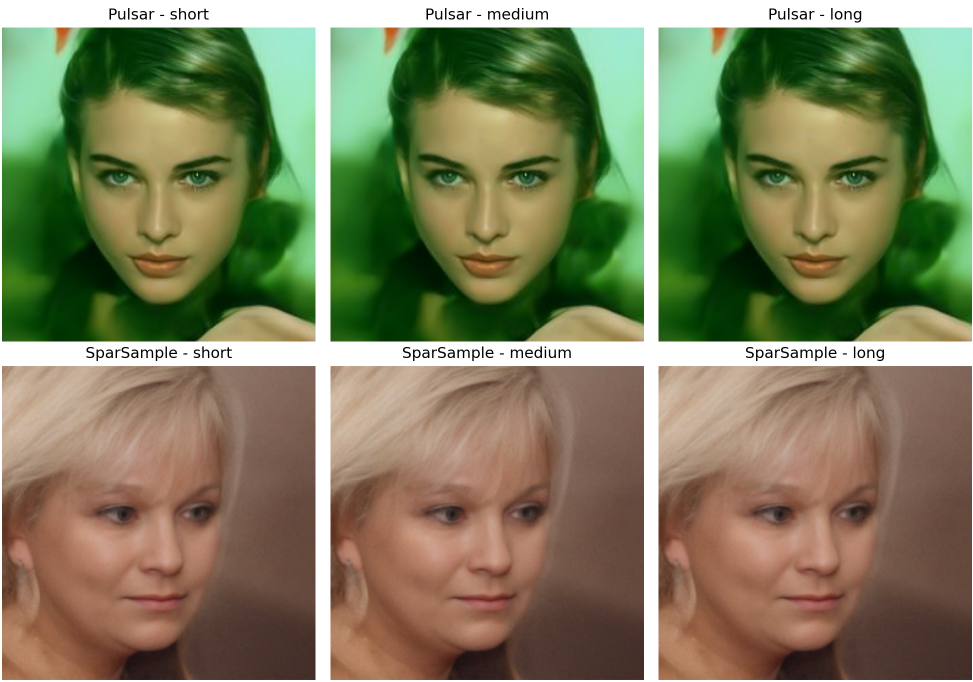
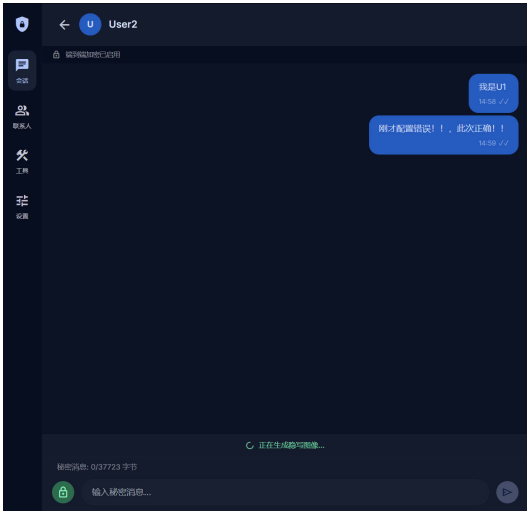
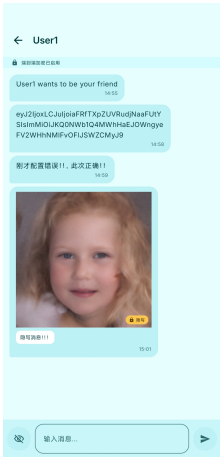


图 5.7 隐写样本对比



(a) Web 端隐写聊天



(b) Android 端隐写聊天

图 5.8 隐写模式聊天效果

5.4 跨平台兼容性测试

系统设计的一个重要目标是支持跨平台通信。测试了 Android 客户端与 Web 客户端之间的互操作性，结果如表 5.9 所示。

表 5.9 跨平台兼容性测试

编号	测试场景	结果
T-50	Android 发送普通消息 → Web 接收	通过
T-51	Web 发送普通消息 → Android 接收	通过
T-52	Android 发送加密消息 → Web 解密	通过
T-53	Web 发送加密消息 → Android 解密	通过
T-54	Android 发送隐写消息 → Web 提取	通过
T-55	Web 发送隐写消息 → Android 提取	通过
T-56	Android 发送好友请求 → Web 处理	通过
T-57	Web 发送好友请求 → Android 处理	通过
T-58	单端登录：Android 登录后 Web 被踢出	通过
T-59	单端登录：Web 登录后 Android 被踢出	通过
T-60	消息状态回执跨平台同步	通过

单端登录机制的测试效果如图 5.9 所示，当用户在 Android 端登录后，Web 端自动弹出提示并跳转至登录页面，验证了单端登录策略的有效性。

测试结果表明，Android 客户端和 Web 客户端之间能够正常互相发送普通消息和隐写消息，端到端加密的跨平台加解密正确，好友管理、消息状态管理和单端登录功能跨平台工作正常。SealedMessage 的 JSON 信封格式和 base64url 编码在两个平台间完全兼容。

5.5 本章小结

本章从功能测试、隐写性能分析和跨平台兼容性三个维度对系统进行了全面测试。功能测试覆盖了用户管理、好友管理、消息生命周期、端到端加密和隐写操作等核心功能，共计 60 个测试用例全部通过。隐写性能分析验证了双算法在安全性和图像质量方面的表现，以及端到端加密对隐写载荷的双重保护。跨平台兼容性测试确认了 Android 和 Web 客户端之间的完整互操作性，包括加密消息、隐写消息和消息状态回执的跨平台同步。测试结果表明系统功能完整、运行稳定。



图 5.9 单端登录效果

6 总结与展望

6.1 工作总结

本文围绕可证安全图像隐写系统的设计与实现展开研究，主要完成了以下工作：

1. **双算法隐写引擎**：集成了 Pulsar 和 SparSamp 两种基于扩散模型的可证安全隐写算法，设计了统一的算法注册与切换机制。Pulsar 基于像素空间扩散模型（DDIM 采样器），通过控制最后一步调度器的方差噪声来源和变码率级联纠错码实现消息嵌入；SparSamp 基于稀疏采样思想，在 IDDPm P2-weighting 框架下通过消息驱动采样和稀疏采样消歧机制实现消息嵌入。两种算法共享相同的 API 接口，支持运行时切换。SparSamp 实现了状态缓存机制，避免重复的扩散采样计算。
2. **端到端加密机制**：实现了基于对称助记词的端到端加密方案。用户通过助记词短语经 PBKDF2-SHA256 (100,000 次迭代) 派生 256 位用户密钥；通信双方的用户密钥按字典序排列后经 HKDF-SHA256 协商出对称消息密钥；消息使用 AES-256-GCM 加密封装为 SealedMessage 格式。消息密钥仅在内存中存在，不进行持久化存储。在隐写聊天模式下，加密后的密文作为隐写嵌入的载荷，实现了“先加密、再隐写”的双重保护。
3. **系统架构设计**：设计了基于客户端-服务端的跨平台隐写通信系统架构。服务端基于 Python FastAPI 框架构建，集成双算法隐写引擎和 WebSocket 实时通信服务；客户端分别使用 Kotlin Jetpack Compose (Android) 和 Vue.js 3 (Web) 开发，实现了完整的用户交互、本地数据管理和端到端加解密。
4. **实时通信系统**：基于 WebSocket 协议实现了全双工实时通信，支持消息即时推送、完整的消息生命周期管理（已发送/已送达/已读回执/消息撤回）、在线状态感知、好友请求通知和离线消息缓存。设计并实现了单端登录安全策略，防止多设备同时登录导致的数据一致性问题。
5. **用户体系与社交功能**：实现了完整的用户注册登录系统（PBKDF2 密码哈希 + JWT 认证）、基于邀请码的好友添加机制，以及好友请求的异步处理流程。支持联系人屏蔽功能，被屏蔽用户的后续消息自动过滤。
6. **隐写通信功能**：在聊天界面中集成了隐写模式，用户可在普通消息和隐写消息之间无缝切换。系统自动使用通信双方的邀请码 XOR 运算派生隐写密钥，实现了端到端的隐蔽信息传输。同时提供了独立的隐写工具页面，支持手动嵌入和

提取操作，支持算法和模型选择。

7. **多平台客户端开发**：Android 客户端采用 Material 3 Expressive 薄荷色主题，包含 9 个页面和 4 个 ViewModel；Web 客户端采用 Aegis Slate 暗色 Material 3 设计系统，包含 10 个页面组件和 4 个 Pinia Store。两个客户端的功能完全对齐，支持跨平台互通。
8. **系统测试验证**：对系统进行了全面的功能测试、隐写性能分析和跨平台兼容性测试，共计 60 个测试用例全部通过，覆盖了用户管理、好友管理、消息生命周期、端到端加密、双算法隐写和跨平台互操作等核心功能。

6.2 不足与展望

尽管本系统已实现了基本完整的可证安全隐写通信功能，但仍存在一些不足之处，有待在后续工作中改进：

1. **嵌入容量有限**：受限于扩散模型的采样步数和图像分辨率，当前系统的单幅图像嵌入容量约为数百字节，仅适合短文本消息的传输。未来可通过支持更大分辨率的模型、优化编码算法或采用多图像分块嵌入策略来提升容量。
2. **计算延迟较高**：两种隐写算法均涉及多步扩散模型推理，单次嵌入或提取操作耗时较长。SparSamp 通过状态缓存机制缓解了重复计算的问题，但首次请求仍需约 50 秒。未来可通过模型量化、蒸馏或使用更高效的扩散模型采样器来降低延迟。
3. **端到端加密方案的局限性**：当前采用的对称助记词方案缺乏前向安全性和未来安全性。未来可考虑引入 Signal 协议的双棘轮算法，实现密钥的自动轮换和前向安全保护。
4. **隐写密钥派生方案的安全性不足**：当前隐写密钥通过双方邀请码 XOR 运算生成 ($K_{\text{stego}} = \text{inviteCode}_A \oplus \text{inviteCode}_B$)，该方案存在明显局限：邀请码仅为 8 位字母数字字符（约 48 比特熵），XOR 运算不增加密钥空间；且邀请码存储在服务端数据库中，服务端管理员或数据库泄露均可推导出隐写密钥。这意味着隐写密钥的安全性依赖于服务端的可信度，与“可证安全”的理想目标存在差距。未来可考虑基于双方助记词通过 HKDF 派生独立的隐写密钥，使其安全性与端到端加密密钥保持一致。
5. **服务端集中式计算**：当前隐写计算完全在服务端执行，存在单点瓶颈和隐私风

险。未来可探索将部分计算迁移到客户端（如使用 WebAssembly 或移动端 GPU 加速），或采用分布式计算架构分担负载。

6. **群组聊天支持**：当前系统仅支持一对一聊天。未来可扩展为群组聊天功能，但需要解决群组密钥管理和多方隐写密钥协商的问题。
7. **好友关系本地存储的局限性**：当前系统的好友关系仅存储在客户端本地数据库中，服务端不维护好友列表。这意味着用户更换设备或清除本地数据后，好友关系将丢失，需要重新添加。未来可在服务端引入加密存储的好友关系表，或利用端到端加密的助记词在新设备上恢复好友关系。
8. **文件类型扩展**：当前系统仅支持文本消息的隐写传输。未来可扩展支持文件、图片等多种数据类型的隐写嵌入，拓展应用场景。

总体而言，本文设计并实现的可证安全图像隐写系统将可证安全隐写理论与端到端加密技术相结合，通过双算法支持和跨平台实现，验证了可证安全隐写技术在实际通信系统中的可行性。系统在功能完整性、跨平台兼容性和通信安全性方面均达到了预期目标，60 个测试用例的通过证明了所设计架构的正确性和实用性。尽管在嵌入容量和计算效率方面仍有提升空间，但本文的工作表明，基于扩散模型的可证安全隐写技术已具备在实际通信场景中部署的基本条件，为安全隐蔽通信领域从理论走向工程应用提供了一种可参考的实现方案。

参考文献

- [1] FRIDRICH J. Steganography in Digital Media: Principles, Algorithms, and Applications[M]. Cambridge University Press, 2009.
- [2] KATZENBEISSER S, PETITCOLAS F A P. Information Hiding: Techniques for Steganography and Digital Watermarking[M]. Artech House, 2000.
- [3] CHAN C K, CHENG L M. Hiding Data in Images by Simple LSB Substitution[J]. Pattern Recognition, 2004, 37(3): 469-474.
- [4] FRIDRICH J, KODOVSKÝ J. Rich Models for Steganalysis of Digital Images[J]. IEEE Transactions on Information Forensics and Security, 2012, 7(3): 868-882.
- [5] BOROUMAND M, CHEN M, FRIDRICH J. Deep Residual Network for Steganalysis of Digital Images[J]. IEEE Transactions on Information Forensics and Security, 2019, 14(5): 1181-1193.
- [6] HOPPER N J, LANGFORD J, VON AHN L. Provably Secure Steganography[J]. IEEE Transactions on Computers, 2009, 58(5): 662-676.
- [7] DE WITT C S, SOKOTA S, KOLTER J Z, et al. Perfectly Secure Steganography Using Minimum Entropy Coupling[C]//Proceedings of the International Conference on Machine Learning (ICML). 2023.
- [8] MARLINSPIKE M, PERRIN T. The Signal Protocol[EB/OL]. 2016. <https://signal.org/docs/>.
- [9] JOIS T M, BECK G, KAPTCHUK G. Pulsar: Secure Steganography for Diffusion Models[C]//Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24). ACM, 2024: 4703-4717.
- [10] MIELIKAİNEN J. LSB Matching Revisited[J]. IEEE Signal Processing Letters, 2006, 13(5): 285-287.
- [11] PEVNÝ T, FILLER T, BAS P. Using High-Dimensional Image Models to Perform Highly Undetectable Steganography[C]//Proceedings of the 12th International Conference on Information Hiding. 2010: 161-177.
- [12] HOLUB V, FRIDRICH J. Designing Steganographic Distortion Using Directional Filters[C]//IEEE International Workshop on Information Forensics and Security (WIFS). 2012: 234-239.
- [13] HOLUB V, FRIDRICH J, DENEMARK T. Universal Distortion Function for Steganography in an Arbitrary Domain[J]. EURASIP Journal on Information Security, 2014, 2014(1): 1-13.
- [14] WESTFELD A. F5—A Steganographic Algorithm[C]//Proceedings of the 4th International Workshop on Information Hiding. 2001: 289-302.
- [15] FRIDRICH J, PEVNÝ T, KODOVSKÝ J. Statistically Undetectable JPEG Steganography: Dead Ends, Challenges, and Opportunities[C]//Proceedings of the 9th ACM Workshop on Multimedia and Security. 2007: 3-14.
- [16] QIAN Y, DONG J, WANG W, et al. Deep Learning for Steganalysis via Convolutional Neural Networks[J]. Media Watermarking, Security, and Forensics, Proceedings of SPIE, 2015, 9409: 94090J.
- [17] XU G, WU H Z, SHI Y Q. Structural Design of Convolutional Neural Networks for Steganalysis[J]. IEEE Signal Processing Letters, 2016, 23(5): 708-712.

- [18] YE J, NI J, YI Y. Deep Learning Hierarchical Representations for Image Steganalysis[J]. IEEE Transactions on Information Forensics and Security, 2017, 12(11): 2545-2557.
- [19] CACHIN C. An Information-Theoretic Model for Steganography[J]. Information and Computation, 2004, 192(1): 41-56.
- [20] HU D, WANG L, JIANG W, et al. A Novel Image Steganography Method via Deep Convolutional Generative Adversarial Networks[J]. IEEE Access, 2018, 6: 38303-38314.
- [21] WANG Y, PEI G, CHEN K, et al. SparSamp: Efficient Provably Secure Steganography Based on Sparse Sampling[C]//Proceedings of the 34th USENIX Conference on Security Symposium. USENIX Association, 2025: 6817-6835.
- [22] SONG J, MENG C, ERMON S. Denoising Diffusion Implicit Models[J]. ArXiv preprint arXiv:2010.02502, 2020.
- [23] HO J, JAIN A, ABBEEL P. Denoising Diffusion Probabilistic Models[J]. Advances in Neural Information Processing Systems, 2020, 33: 6840-6851.
- [24] ROMBACH R, BLATTMANN A, LORENZ D, et al. High-Resolution Image Synthesis with Latent Diffusion Models[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2022: 10684-10695.
- [25] NICHOL A, DHARIWAL P. Improved Denoising Diffusion Probabilistic Models[C]//Proceedings of the 38th International Conference on Machine Learning (ICML). 2021: 8162-8171.
- [26] KRAWCZYK H, ERONEN P. HMAC-based Extract-and-Expand Key Derivation Function (HKDF)[Z]. RFC 5869. 2010.
- [27] DWORKIN M. Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC[Z]. NIST Special Publication 800-38D. 2007.
- [28] RAMÍREZ S. FastAPI: Modern, Fast (High-Performance), Web Framework for Building APIs with Python[EB/OL]. 2019. <https://fastapi.tiangolo.com/>.
- [29] FETTE I, MELNIKOV A. The WebSocket Protocol[Z]. RFC 6455. 2011.
- [30] Google. Jetpack Compose: Android's Modern Toolkit for Building Native UI[EB/OL]. 2021. <https://developer.android.com/jetpack/compose>.
- [31] YOU E. Vue.js: The Progressive JavaScript Framework[EB/OL]. 2020. <https://vuejs.org/>.

致谢

首先，我要向我的指导教师王垚飞副教授致以最诚挚的感谢。从论文选题、方案设计到系统实现和论文撰写，王老师始终给予我悉心的指导和耐心的帮助。王老师严谨的治学态度、深厚的学术素养和对学生负责任的精神，使我在本科阶段受益匪浅，也为本文的顺利完成奠定了坚实的基础。

感谢合肥工业大学信息安全专业全体老师四年来的辛勤教导。各位老师在密码学、网络安全、软件工程等课程中传授的专业知识，为本课题的研究提供了必要的理论基础和技术储备。感谢学院提供的实验环境和学习资源，使我能够在良好的条件下完成毕业设计。

感谢实验室的各位同学在学习和生活中给予的帮助与支持。在课题研究过程中，与同学们的讨论和交流让我获得了许多有价值的启发。特别感谢在系统开发和调试阶段给予技术建议的同学们，你们的帮助使许多难题得以顺利解决。

最后，衷心感谢我的家人。感谢父母多年来的养育之恩和无私付出，你们的理解、鼓励和支持是我完成学业的最大动力。你们的关爱始终伴随着我的成长，给予我面对困难的勇气和力量。

作者：支雅安

2026 年 5 月 29 日

附录

附录 A 端到端加密核心代码

A.1 SealedMessage 信封实现 (Kotlin)

```
object SealedMessage {  
    private const val VERSION = 1  
    private const val NONCE_BYTES = 12  
    private const val GCM_TAG_BITS = 128  
  
    fun seal(mkey: ByteArray, plaintext: String): String {  
        val nonce = ByteArray(NONCE_BYTES)  
        SecureRandom().nextBytes(nonce)  
        val cipher = Cipher.getInstance("AES/GCM/NoPadding")  
        val spec = GCMParameterSpec(GCM_TAG_BITS, nonce)  
        cipher.init(Cipher.ENCRYPT_MODE, SecretKeySpec(mkey, "AES"), spec)  
        val ct = cipher.doFinal(plaintext.toByteArray(Charsets.UTF_8))  
        val json = "" + {"v": $VERSION,  
            "n": "${b64uEncode(nonce)}",  
            "c": "${b64uEncode(ct)}"}  
        return b64uEncode(json.toByteArray(Charsets.UTF_8))  
    }  
  
    fun tryOpen(mkey: ByteArray, sealed: String): String? {  
        return try {  
            val outer = String(b64uDecode(sealed), Charsets.UTF_8)  
            val obj = JSONObject(outer)  
            if (obj.optInt("v") != VERSION) return null  
            val nonce = b64uDecode(obj.getString("n"))  
            val ct = b64uDecode(obj.getString("c"))  
            val cipher = Cipher.getInstance("AES/GCM/NoPadding")  
            val spec = GCMParameterSpec(GCM_TAG_BITS, nonce)  
            cipher.init(Cipher.DECRYPT_MODE, SecretKeySpec(mkey, "AES"), spec)  
            String(cipher.doFinal(ct), Charsets.UTF_8)  
        } catch (_: Exception) { null }  
    }  
}
```

A.2 密钥派生实现 (Kotlin)

```

object CryptoUtils {
    private const val SEED_SALT = "stegochat-seed-v1"
    private const val MKEY_SALT = "stegochat-mkey-v1"
    private const val MKEY_INFO = "e2ee"
    private const val PBKDF2_ITER = 100_000
    private const val USER_KEY_BITS = 256
    private const val MKEY_BYTES = 32
    private const val FINGERPRINT_BYTES = 8

    fun deriveUserKey(phrase: String): ByteArray {
        val factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256")
        val spec = PBEKeySpec(
            phrase.toCharArray(),
            SEED_SALT.toByteArray(Charsets.UTF_8),
            PBKDF2_ITER, USER_KEY_BITS
        )
        return factory.generateSecret(spec).encoded
    }

    fun deriveMkey(selfKey: ByteArray, peerKey: ByteArray): ByteArray {
        val (lo, hi) = if (selfKey.toHexString() <= peerKey.toHexString())
            selfKey to peerKey else peerKey to selfKey
        val ikm = lo + hi
        return hkdfSha256(
            ikm = ikm,
            salt = MKEY_SALT.toByteArray(Charsets.UTF_8),
            info = MKEY_INFO.toByteArray(Charsets.UTF_8),
            length = MKEY_BYTES
        )
    }

    fun fingerprint(userKey: ByteArray): String {
        val hash = MessageDigest.getInstance("SHA-256").digest(userKey)
        return hash.take(FINGERPRINT_BYTES)
            .joinToString(":") { "%02x".format(it) }
    }
}

```

附录 B WebSocket 连接管理核心代码

B.1 服务端 ConnectionManager

```

class ConnectionManager:
    def __init__(self):
        self.active: dict[str, WebSocket] = {}

    async def connect(self, user_id: str, ws: WebSocket):

```

```

        await ws.accept()
        old = self.active.get(user_id)
        if old:
            try:
                await old.send_text(json.dumps({
                    "type": "kicked",
                    "payload": {
                        "reason": "Logged in on another device"
                    }
                }))
            except Exception:
                pass
        async def _force_close():
            await asyncio.sleep(5)
            try:
                await old.close(code=4001)
            except Exception:
                pass
        asyncio.create_task(_force_close())
        self.active[user_id] = ws

    async def send_to_user(self, user_id: str,
                           message: dict) -> bool:
        ws = self.active.get(user_id)
        if ws:
            try:
                await ws.send_text(json.dumps(message))
                return True
            except Exception:
                self.disconnect(user_id, ws)
        return False

```

附录 C SparSamp 核心算法代码

C.1 消息驱动采样与稀疏采样嵌入

```

def gaussian_quantize_256(mu, sigma, low=-1.0, high=1.0):
    """将 N(mu, sigma) 量化为 256 个 bin 的概率分布"""
    edges = np.linspace(low, high, 257)
    cdf = norm.cdf(edges, loc=mu, scale=sigma)
    probs = np.diff(cdf)
    probs = np.clip(probs, 1e-30, None)
    return probs / probs.sum()

def encode_step(bin_idx, probs, n_m, k_m, block_size):
    """单像素消息驱动采样步骤"""
    r = random.random()
    cum = np.cumsum(probs)

```

```
r_i_m = (k_m / n_m + r) % 1.0
selected = int(np.searchsorted(cum, r_i_m))
selected = min(selected, len(probs) - 1)

cum_full = np.concatenate(([0.0], cum))
lo = cum_full[selected]
hi = cum_full[selected + 1]
n_m_new = int(np.floor(n_m * (hi - lo)))
k_m_new = int(np.floor(k_m - n_m * lo))
n_m_new = max(n_m_new, 1)

done = (n_m_new <= 1)
return selected, n_m_new, k_m_new, done
```